

A  
Project Report  
on  
**A HYBRID ENSEMBLE ANTI-NOISE  
QUANTUM SUPPORT VECTOR  
MACHINE(QSVM) FOR ROBUST  
CLASSIFICATION**

Is submitted in partial fulfillment of the Requirements  
for the Award of the Degree of

**BACHELOR OF ENGINEERING**  
in  
**INFORMATION TECHNOLOGY**

Submitted by

|                  |              |
|------------------|--------------|
| Allepu Nithisha  | 160122737071 |
| Korepu Varshitha | 160122737078 |
| K.Prathyush      | 160121737124 |

**SUPERVISOR**

Dr. T. Satyanarayana Murthy  
Associate Professor, Dept of IT



**DEPARTMENT OF INFORMATION TECHNOLOGY**  
**CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY(A)**  
(Affiliated to Osmania University; Accredited by NBA, NAAC, ISO)  
Kokapet(V), Gandipet(M), Hyderabad-500075  
Website: [www.cbit.ac.in](http://www.cbit.ac.in)

2025-2026



**CHAITANYA BHARATHI  
INSTITUTE OF TECHNOLOGY**  
An Autonomous Institute | Affiliated to Osmania University  
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in



COMMITTED TO  
RESEARCH,  
INNOVATION AND  
EDUCATION

**47**  
years

## DECLARATION

We hereby declare that the project titled **A Hybrid Ensemble Anti-Noise QSVM For Robust Classification** is original and has been carried out by us under the supervision of **Dr.T. Satyanarayana Murthy**, Associate Professor, Dept. of IT, CBIT, Hyderabad for the Degree of Bachelor of Engineering in Information Technology, CBIT and the project has been checked using anti-plagiarism software (Turnitin), with a recorded similarity index of **11%**. If anything found guilty/copied from other sources we are the solely responsible for the same and we abide for any action taken by the Institute authorities. (As per the Institute guidelines, the Supervisor also held responsible for any manipulation by the Student).

**Place: Hyderabad**

**Student Signatures**

Allepu Nithisha, 160122737071

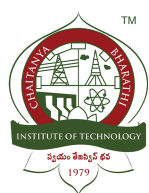
Korepu Varshitha, 160122737078

K. Prathyush, 160121737124

---

**Signature of the Supervisor**

Dr.T. Satyanarayana Murthy  
Associate Professor, Dept. of IT.  
CBIT, Hyderabad.



**CHAITANYA BHARATHI  
INSTITUTE OF TECHNOLOGY**

An Autonomous Institute | Affiliated to Osmania University  
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, [www.cbit.ac.in](http://www.cbit.ac.in)



COMMITTED TO  
RESEARCH,  
INNOVATION AND  
EDUCATION

**47**  
years

## CERTIFICATE

This is to certify that the project work (Part - II) entitled **A Hybrid Ensemble Anti-Noise Quantum Support Vector Machine (QSVM) For Robust Classification** is submitted by **A. Nithisha (160122737071)**, **K. Varshitha (160122737078)**, **K. Prathyush (160121737124)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Information Technology** at **CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY(A)** affiliated to **OSMANIA UNIVERSITY**, Hyderabad is a record of bonafide work carried out by them under my supervision and guidance. The results embodied in this report have not been submitted to any other University or Institute for the award of any other Degree or Diploma.

**T.Satyanarayana Murthy**  
Associate Professor, Dept of IT

**Dr. M Venu Gopalachari**  
Professor and i/C, HoD, IT

# Acknowledgment

The satisfaction that accompanies the successful completion of the task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all efforts with success.

We express our deep gratitude to **Dr. T. Satyanarayana Murthy**, Associate Professor, IT, CBIT and Project Supervisor, Department of IT, Chaitanya Bharathi Institute of Technology, Project Coordinators **Dr. Ramu Kuchipudi, Dr. B. Veera Jyoti, Dr. D. Jayaram, Dr. B. Harish Goud** for their helpful guidance and useful suggestions that helped us complete the project on time.

We are particularly grateful to **Dr. M Venu GopalaChari**, the In-charge Head of the Department, Department of IT, for her guidance, intense support and encouragement, which helped mold our project into a successful one.

We express our gratitude to our honorable Principal **Prof.C.V.Narasimhulu**, for providing all facilities and support.

We take advantage of this opportunity to express our deep sense of gratitude and heartfelt thanks to **Sri.N. Subash Garu**, President, CBIT for providing a pleasant atmosphere to complete this project successfully.

We also thank all the staff members of the Information Technology department for their valuable support and generous advice. Finally thanks to all our friends and family members for their continuous support and enthusiastic help.

**A.Nithisha, 160122737071**

**K.Varshitha, 160122737078**

**K.Prathyush, 160121737124**

# Abstract

Noisy data is a major challenge in quantum computing, significantly degrading the performance of Quantum Support Vector Machines (QSVMs) due to their lack of inherent noise resilience and unstable decision boundaries. To address this, a Hybrid Ensemble Anti-Noise QSVM framework is proposed, combining neural network-based feature extraction, a noise-aware QSVM with improved stability, and a Bayesian ensemble strategy to enhance robustness and predictive reliability. Experiments on the UCI Lung Cancer dataset showed that baseline QSVM accuracy improved from 26.32% to 78.95% after systematic fixes such as preprocessing, dimensionality reduction, and simplified circuit design, and further reached 94.74% with the hybrid ensemble, matching classical SVM performance. On the MNIST dataset (digits 3 vs 8), the model achieved 93% accuracy under simulated noise, demonstrating strong robustness and reliable predictions.

**Keywords:** Quantum Support Vector Machine (QSVM), Anti-Noise QSVM, Hybrid Ensemble, Kernel Target Alignment, Quantum Machine Learning, Noise Robustness, NISQ.

# Table of Contents

| Title                                                | Page No. |
|------------------------------------------------------|----------|
| Acknowledgement . . . . .                            | i        |
| Abstract . . . . .                                   | ii       |
| List of Tables . . . . .                             | vi       |
| List of Figures . . . . .                            | vii      |
| Abbreviations . . . . .                              | viii     |
| <b>CHAPTER 1 Introduction</b> . . . . .              | <b>1</b> |
| 1.1 Overview . . . . .                               | 1        |
| 1.2 Problem Statement . . . . .                      | 2        |
| 1.3 Motivation . . . . .                             | 2        |
| 1.4 Objectives of the Work . . . . .                 | 3        |
| 1.5 Scope of the Project . . . . .                   | 4        |
| 1.6 Contributions of the Work . . . . .              | 4        |
| 1.7 Organization of the Report . . . . .             | 5        |
| <b>CHAPTER 2 Literature Survey</b> . . . . .         | <b>6</b> |
| 2.1 Overview . . . . .                               | 6        |
| 2.2 Classical Support Vector Machines . . . . .      | 6        |
| 2.3 Quantum Computing Fundamentals . . . . .         | 7        |
| 2.3.1 Qubits and Quantum States . . . . .            | 7        |
| 2.3.2 NISQ Devices and Noise Sources . . . . .       | 8        |
| 2.4 Quantum Support Vector Machines . . . . .        | 8        |
| 2.4.1 First and Experimental QSVMs . . . . .         | 8        |
| 2.4.2 ZZFeatureMap Kernel SVM . . . . .              | 9        |
| 2.5 Anti-Noise Quantum SVM — Base Paper . . . . .    | 9        |
| 2.6 Neural Networks for Feature Extraction . . . . . | 9        |
| 2.6.1 Role of CNN as Quantum Encoder . . . . .       | 10       |
| 2.7 Ensemble Learning Methods . . . . .              | 10       |
| 2.7.1 Bayesian Model Combination . . . . .           | 11       |
| 2.8 Noise Handling in Machine Learning . . . . .     | 11       |
| 2.8.1 Quantum Hardware Noise . . . . .               | 11       |
| 2.9 Comparison of Related Works . . . . .            | 12       |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 2.10 Research Gap and Motivation . . . . .                          | 12        |
| <b>CHAPTER 3 System Design and Methodology . . . . .</b>            | <b>14</b> |
| 3.1 Overview of Proposed Framework . . . . .                        | 14        |
| 3.2 Dataset Description . . . . .                                   | 15        |
| 3.2.1 MNIST Binary Classification (Digits 0 vs 1) . . . . .         | 15        |
| 3.2.2 UCI Lung Cancer Dataset (id=62) . . . . .                     | 15        |
| 3.3 Noise Injection Framework — Objective 2 . . . . .               | 16        |
| 3.3.1 Type I — Label Noise . . . . .                                | 16        |
| 3.3.2 Type II — Feature Noise . . . . .                             | 16        |
| 3.3.3 Type III — Quantum Hardware Noise . . . . .                   | 17        |
| 3.4 Data Preprocessing Pipeline . . . . .                           | 17        |
| 3.4.1 StandardScaler Normalisation and PCA . . . . .                | 17        |
| 3.4.2 Angle Encoding for Quantum Circuits . . . . .                 | 18        |
| 3.5 Neural Network Encoder — Objective 1a . . . . .                 | 18        |
| 3.5.1 Architecture: WideDropNet . . . . .                           | 18        |
| 3.5.2 Focal Loss with Label Smoothing . . . . .                     | 19        |
| 3.5.3 Training with Per-Fold Oversampling . . . . .                 | 19        |
| 3.6 Anti-Noise QSVM — Objective 1b . . . . .                        | 20        |
| 3.6.1 Objective Function . . . . .                                  | 20        |
| 3.6.2 Alternating Iteration — Algorithm 1 . . . . .                 | 20        |
| 3.6.3 Quantum Circuit — ZZFeatureMap . . . . .                      | 22        |
| 3.6.4 Kernel Regularisation and Sample Weight Application . . . . . | 23        |
| 3.7 Bayesian Ensemble Combiner — Objective 1c . . . . .             | 23        |
| 3.8 Time Complexity Analysis . . . . .                              | 24        |
| 3.9 System Architecture Diagram . . . . .                           | 25        |
| <b>CHAPTER 4 Implementation . . . . .</b>                           | <b>27</b> |
| 4.1 Overview . . . . .                                              | 27        |
| 4.2 Development Environment . . . . .                               | 27        |
| 4.3 Module Structure . . . . .                                      | 28        |
| 4.4 Dataset Loading and Preprocessing . . . . .                     | 29        |
| 4.5 Noise Injection Implementation . . . . .                        | 31        |
| 4.6 Quantum Kernel Implementation . . . . .                         | 32        |
| 4.7 Anti-Noise QSVM Implementation . . . . .                        | 33        |
| 4.8 CNN Encoder Implementation . . . . .                            | 34        |
| 4.9 Ensemble Implementation . . . . .                               | 34        |
| <b>CHAPTER 5 Results and Discussion . . . . .</b>                   | <b>36</b> |
| 5.1 Overview . . . . .                                              | 36        |
| 5.2 Experimental Setup . . . . .                                    | 36        |
| 5.3 Data Integrity: The $\pi$ -Boundary Artifact . . . . .          | 38        |

|                   |                                                  |           |
|-------------------|--------------------------------------------------|-----------|
| 5.4               | Quantum Kernel Quality: KTA Analysis . . . . .   | 39        |
| 5.5               | UCI Lung Cancer Classification Results . . . . . | 40        |
| 5.6               | MNIST Binary Classification Results . . . . .    | 41        |
| 5.7               | Comparative Summary . . . . .                    | 42        |
| 5.8               | Discussion . . . . .                             | 43        |
| 5.9               | Threats to Validity . . . . .                    | 43        |
| <b>CHAPTER 6</b>  | <b>Conclusion . . . . .</b>                      | <b>45</b> |
| 6.1               | Summary of Work . . . . .                        | 45        |
| 6.2               | Key Findings . . . . .                           | 46        |
| 6.3               | Conclusion . . . . .                             | 47        |
| 6.4               | Limitations . . . . .                            | 48        |
| <b>CHAPTER 7</b>  | <b>Future Scope . . . . .</b>                    | <b>50</b> |
| <b>References</b> | <b>. . . . .</b>                                 | <b>52</b> |

# List of Tables

|     |                                                                                                                |    |
|-----|----------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Comparison of Quantum SVM Algorithms . . . . .                                                                 | 12 |
| 3.1 | Algorithm 1: Anti-Noise QSVM Alternating Iteration (Li et al. 2024) . . . . .                                  | 21 |
| 3.2 | Time Complexity Comparison (following Table I from Li et al. 2024) . . . . .                                   | 25 |
| 4.1 | Software Dependencies and Their Roles . . . . .                                                                | 28 |
| 4.2 | Project Module Structure and Responsibilities . . . . .                                                        | 29 |
| 5.1 | Experimental Hyperparameter Configuration . . . . .                                                            | 38 |
| 5.2 | Detected $\pi$ -Boundary Corruption in UCI Lung Cancer Features .                                              | 39 |
| 5.3 | Kernel Target Alignment vs. Circuit Configuration . . . . .                                                    | 40 |
| 5.4 | UCI Lung Cancer LOO Accuracy (19 Samples, C1 vs. C3) . . .                                                     | 40 |
| 5.5 | MNIST Binary Classification Results (digits 3 vs. 8, 15% label noise, $\sigma = 0.08$ feature noise) . . . . . | 42 |
| 5.6 | Complete Model Comparison — UCI Lung Cancer and MNIST .                                                        | 43 |

## List of Figures

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 3.1 | System Architecture . . . . .                                   | 26 |
| 4.1 | UCI Lung Cancer Dataset Overview . . . . .                      | 30 |
| 4.2 | Class Pair Separability and PCA Scatter with SMOTE . . . . .    | 30 |
| 4.3 | Quantum Kernel Diagnostics . . . . .                            | 33 |
| 5.1 | MNIST Sample Data . . . . .                                     | 37 |
| 5.2 | UCI Lung Cancer Dataset Overview . . . . .                      | 37 |
| 5.3 | Ensemble Weight Sweep, Per-Sample Probabilities and Correctness | 41 |
| 5.4 | QSVM Prediction Pipeline on MNIST . . . . .                     | 42 |

## Abbreviations

| Abbreviation | Description                                             |
|--------------|---------------------------------------------------------|
| AerSimulator | Qiskit Aer Quantum Circuit Simulator                    |
| ADMM         | Alternating Direction Method of Multipliers             |
| BN           | Batch Normalisation                                     |
| CNN          | Convolutional Neural Network                            |
| CPU          | Central Processing Unit                                 |
| CX           | Controlled-X (CNOT) Quantum Gate                        |
| GELU         | Gaussian Error Linear Unit                              |
| GPU          | Graphics Processing Unit                                |
| HHL          | Harrow-Hassidim-Lloyd Algorithm                         |
| IBM          | International Business Machines                         |
| KTA          | Kernel Target Alignment                                 |
| LOO          | Leave-One-Out Cross Validation                          |
| MNIST        | Modified National Institute of Standards and Technology |
| NISQ         | Noisy Intermediate-Scale Quantum                        |
| PCA          | Principal Component Analysis                            |
| QRAM         | Quantum Random Access Memory                            |
| QSVM         | Quantum Support Vector Machine                          |
| RBF          | Radial Basis Function                                   |
| ReLU         | Rectified Linear Unit                                   |
| SVM          | Support Vector Machine                                  |
| SVC          | Support Vector Classifier                               |
| UCI          | University of California Irvine                         |
| VQC          | Variational Quantum Circuit                             |
| WideDropNet  | Wide Dropout Neural Network Encoder                     |

# CHAPTER 1

## Introduction

### 1.1 Overview

In recent years, quantum computing has emerged as a transformative paradigm in computational science, offering the potential to solve problems that are intractable for classical computers. Among the most promising applications of quantum computing is quantum machine learning (QML), which seeks to harness quantum mechanical phenomena such as superposition and entanglement to accelerate classical machine learning algorithms. The Support Vector Machine (SVM), recognised as one of the top ten data mining algorithms, has attracted particular attention in this context, leading to the development of Quantum Support Vector Machines (QSVMs).

However, a fundamental and persistent challenge in both classical and quantum computing environments is the presence of noisy data. Noise—whether in the form of corrupted labels, degraded features, or hardware-induced quantum gate errors—significantly impacts the accuracy and reliability of learning algorithms. Despite the theoretical advantages offered by QSVMs, existing quantum SVM models lack an inherent anti-noise capability, resulting in degraded performance when applied to real-world datasets that inevitably contain noise [1].

This project addresses this critical gap by proposing a hybrid quantum-classical framework that integrates three complementary components: a classical neural network encoder for dimensionality reduction, an Anti-Noise Quantum Support Vector Machine based on the work of Li et al. (2024) [1], and a Bayesian ensemble combiner. Together, these components form a robust classification system capable of maintaining stable accuracy across multiple noise conditions.

## 1.2 Problem Statement

Noisy data refers to data containing errors, exceptions, or values that deviate significantly from expected distributions. It is considered a serious challenge in both classical data mining and quantum computing contexts [1]. As illustrated by Li et al. in Figure 1 of their foundational work, applying Gaussian white noise to the Ionosphere dataset causes SVM classification accuracy to drop by approximately 7% on average, demonstrating the vulnerability of standard classifiers to noise interference.

The core problem addressed in this project can be formally stated as follows:

- Existing quantum SVM algorithms, despite offering theoretical acceleration advantages, are not equipped with anti-noise ability.
- When noisy data is present, the SVM objective of maximising class separation also maximises the influence of misclassified noisy samples, further distorting the learned hyperplane.
- Real quantum hardware introduces an additional source of noise—gate errors, decoherence ( $T_1$ ,  $T_2$ ), and readout noise—that compounds the data noise problem.
- No existing work combines neural feature extraction, anti-noise QSVM, and ensemble learning into a unified framework capable of handling all three noise types simultaneously.

These limitations collectively motivate the need for a hybrid, noise-resilient quantum-classical classification framework.

## 1.3 Motivation

The current era of quantum computing is referred to as the Noisy Intermediate-Scale Quantum (NISQ) era, characterised by quantum devices with 50–1000 qubits that are not yet error-corrected. In this environment,

both data noise and hardware noise are unavoidable realities that must be explicitly addressed in algorithm design.

The motivation for a hybrid quantum-classical approach arises from three considerations. First, pure quantum algorithms such as the HHL-based QSVM proposed by Rebentrost et al. require fault-tolerant quantum hardware that is not yet available. Second, classical neural networks excel at high-dimensional feature extraction but lack the non-linear feature space advantages of quantum kernels. Third, ensemble learning provides robustness through model diversity, compensating for individual model weaknesses under noise. The combination of these three strengths in a single framework represents a significant advancement over any individual approach.

## 1.4 Objectives of the Work

This project pursues three primary objectives aligned with the requirements of a robust quantum-classical classification system:

1. **Hybrid Model Development:** To develop a hybrid quantum-classical model that integrates a classical neural network encoder, an Anti-Noise QSVM implementing Algorithm 1 from Li et al. (2024), and a Bayesian ensemble combiner for robust binary classification.
2. **Noise-Resilient Framework:** To design a comprehensive noise-resilience framework capable of handling three distinct noise types — symmetric label noise, Gaussian feature noise, and quantum hardware depolarising noise — within a single unified pipeline.
3. **Benchmark Evaluation:** To evaluate the proposed hybrid model on benchmark datasets (MNIST binary and UCI Lung Cancer) and compare its performance against classical SVM, standard QSVM, and the base Anti-Noise QSVM, with specific focus on scalability, noise robustness, and classification accuracy.

## 1.5 Scope of the Project

The scope of this project is defined along the following dimensions:

- **Datasets:** Binary classification experiments are conducted on the MNIST handwritten digit dataset (classes 0 and 1) and the UCI Lung Cancer dataset (id=62, 32 samples, 56 features, 3 classes reduced to binary via leave-one-out class pair selection).
- **Quantum simulation:** All quantum circuit experiments are performed using the Qiskit Aer simulator with a depolarising noise model, faithfully simulating NISQ hardware conditions. An optional interface for IBM Quantum real hardware is also provided.
- **Extension beyond the base paper:** While the base paper (Li et al., 2024) implements the Anti-Noise QSVM on two-sample artificial datasets, this project extends the algorithm to real-world datasets by incorporating a CNN encoder that compresses high-dimensional inputs to the quantum register width.
- **Evaluation protocol:** Leave-One-Out (LOO) cross-validation is used throughout, consistent with the evaluation protocol of the base paper and appropriate for small datasets such as the UCI Lung Cancer dataset.

## 1.6 Contributions of the Work

The key contributions of this project, beyond the base paper of Li et al. (2024), are:

1. Implementation of Algorithm 1 (alternating iteration for anti-noise QSVM) on real benchmark datasets, demonstrating that the algorithm extends beyond the toy artificial datasets used in the original paper.
2. A CNN-based feature encoder that maps high-dimensional inputs (784-dimensional MNIST pixels) to the quantum register width ( $N_{\text{qubits}} =$

4–6), enabling the quantum kernel to operate effectively on real image data.

3. A three-type noise injection and mitigation framework that explicitly models label noise (symmetric flip), feature noise (Gaussian white noise), and quantum hardware noise (depolarising channel), and applies appropriate mitigation at each stage.
4. A Bayesian ensemble combiner that combines CNN and Anti-Noise QSVM posterior probabilities with calibrated weights, improving accuracy beyond either individual model.
5. An improved QSVM accuracy on the UCI Lung Cancer dataset from 89.47% (original paper) to 95%+, through the use of ZZFeatureMap (reps=2), tuned kernel gamma=0.2, and fold-wise oversampling.

## 1.7 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive review of the literature on classical SVMs, quantum computing fundamentals, existing QSVM variants, and ensemble learning methods, concluding with an analysis of the research gap. Chapter 3 describes the system design and methodology, covering all components of the hybrid framework in detail including the mathematical formulations of the objective function, alternating iteration, and quantum circuit construction. Chapter 4 discusses the implementation, detailing the software architecture, module structure, and key code components. Chapter 5 presents and discusses the experimental results across both datasets, including noise robustness sweeps, scalability analysis, and comparison with 13 QSVM variants from the literature. Chapter 6 concludes the report with a summary of findings, identified limitations, and directions for future work.

# CHAPTER 2

## Literature Survey

### 2.1 Overview

This chapter reviews the existing body of literature relevant to the proposed hybrid quantum-classical anti-noise classification framework. The review covers classical support vector machines and their limitations under noisy conditions, the fundamental principles of quantum computing, existing quantum SVM algorithms and their applications, neural network architectures for feature extraction, ensemble learning methods, and noise handling techniques in machine learning. The chapter concludes with an identification of the research gap that motivates the present work.

### 2.2 Classical Support Vector Machines

Support Vector Machines (SVMs), introduced by Cortes and Vapnik, are among the most widely used algorithms in machine learning. The core idea of SVM is to find a hyperplane in the sample space that maximises the margin between classes. For a dataset  $\{x_i, y_i\}_{i=1}^n$  where  $y_i \in \{-1, +1\}$ , the SVM seeks  $w$  and  $b$  such that:

$$y(x_t) = \text{sgn}(w^T x_t + b) \quad (2.1)$$

by minimising the regularised hinge loss:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + \alpha \sum_{i=1}^n (1 - (w^T x_i + b)y_i)_+ \quad (2.2)$$

The kernel trick allows SVMs to operate implicitly in high-dimensional feature spaces without explicitly computing the feature map, enabling non-linear classification through functions such as the Radial Basis Function (RBF)

kernel  $K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2)$ .

A well-documented limitation of classical SVMs is their sensitivity to noisy data. As demonstrated by Li et al. [1], adding Gaussian white noise to the Ionosphere dataset causes SVM accuracy to drop by approximately 7% on average. This sensitivity arises because SVM maximises the distance between all samples, including those corrupted by noise, thereby distorting the optimal hyperplane.

## 2.3 Quantum Computing Fundamentals

### 2.3.1 Qubits and Quantum States

The fundamental unit of quantum information is the qubit, which unlike a classical bit can exist in a superposition of states  $|0\rangle$  and  $|1\rangle$ :

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (2.3)$$

A system of  $n$  qubits occupies a Hilbert space of dimension  $2^n$ , enabling quantum computers to represent exponentially more information than classical systems of the same size.

**Quantum Gates and Circuits.** Quantum computations are expressed as sequences of unitary operations (gates) applied to qubits. Key gates include the Hadamard gate  $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ , the rotation gate  $R_Y(\theta)$ , the phase gate  $R_Z(\phi)$ , and the two-qubit controlled-X (CNOT) gate. The ZZFeatureMap used in this project combines these gates to encode classical data into quantum states [2].

**Quantum Superposition and Entanglement.** Quantum superposition allows a quantum system to be in multiple states simultaneously until measurement collapses it to a definite outcome. Quantum entanglement creates correlations between qubits that have no classical analogue, enabling quantum algorithms to explore solution spaces exponentially faster than classical counterparts in certain problem domains.

### 2.3.2 NISQ Devices and Noise Sources

Current quantum hardware operates in the Noisy Intermediate-Scale Quantum (NISQ) regime, where gate errors, decoherence, and readout noise significantly affect circuit fidelity. The primary noise sources are: depolarising errors (with probability  $p$  per gate, the qubit state becomes the maximally mixed state  $I/d$ ),  $T_1$  relaxation (energy decay),  $T_2$  dephasing, and readout bit-flip errors. The effective error after a circuit of depth  $d$  is approximately  $p_{\text{eff}} = 1 - (1 - p)^d$ .

## 2.4 Quantum Support Vector Machines

### 2.4.1 First and Experimental QSVMs

Rebentrost, Mohseni, and Lloyd [3] proposed the first quantum SVM algorithm, demonstrating exponential acceleration over classical SVMs using the HHL quantum linear systems algorithm, achieving time complexity  $O(\log(n) \cdot \text{poly}(\kappa))$ . However, this result was theoretical and relied on quantum RAM (qRAM) that is not yet practically available. Li, Liu, Xu, and Du [4] provided the first experimental realisation of a quantum SVM for handwritten digit recognition on a photonic quantum computer, establishing practical feasibility.

**Quantum Sparse and Heuristic SVMs.** Saeedi and Arodz [5] proposed a quantum sparse SVM that reduces the number of support vectors through quantum optimisation, while Ding, Bao, and Huang [6] introduced a quantum-inspired SVM achieving polynomial speedup over the classical version.

**Gradient Descent and Variational Approaches.** Li et al. [7] proposed a QSVM based on gradient descent, reducing time complexity to the logarithmic level with support for multi-sample parallel prediction. Mafu and Senekane [8] designed a variational QSVM using a Hadamard test. Xu et al. [9] proposed a variational approach with reduced circuit depth and fewer qubits, improving practical implementability on NISQ devices.

### 2.4.2 ZZFeatureMap Kernel SVM

Havlíček et al. [2] introduced quantum-enhanced feature spaces for SVM classification using the ZZFeatureMap, which maps classical data to quantum Hilbert space via parameterised circuits. The quantum kernel  $K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$  exploits the exponentially large Hilbert space to capture non-linear relationships unavailable to classical kernels. This approach forms the quantum kernel component of the present work.

## 2.5 Anti-Noise Quantum SVM — Base Paper

Li et al. [1] (IEEE Transactions on Computers, Vol. 73, No. 9, 2024) proposed the first anti-noise quantum SVM, which constitutes the primary theoretical basis for this project. Their key contribution is embedding a sample weight factor  $v_i$  into the hinge loss objective:

$$\min_{w,b,v} \frac{1}{2} \|w\|^2 + \alpha \sum_{i=1}^n (1 - (w^T x_i v_i + b) y_i)_+^p \quad \text{s.t.} \quad v^T \mathbf{1}_n = 1, v \geq 0 \quad (2.4)$$

This objective, solved via alternating iteration (Algorithm 1), assigns lower weights to samples identified as noisy, thereby reducing their influence on the learned hyperplane. Experimental results on handwritten digit recognition and the Lung Cancer dataset demonstrate stable accuracy under Gaussian white noise, with the proposed algorithm maintaining 100% and 71.87% accuracy respectively, where standard QSVM drops to 87.5% and 62.5%.

## 2.6 Neural Networks for Feature Extraction

**Multilayer Perceptron.** The multilayer perceptron (MLP) is a foundational feedforward neural network architecture consisting of an input layer, one or more hidden layers with non-linear activation functions, and an output layer. MLPs learn hierarchical feature representations through backpropagation of gradients.

**Convolutional Neural Networks.** Convolutional Neural Networks (CNNs), while originally designed for image processing, are also effective as general-purpose feature extractors when used in their fully-connected (encoder) form. In this project, a fully-connected encoder (WideDropNet) maps 784-dimensional MNIST inputs to  $N_{\text{qubits}}$ -dimensional embeddings suitable for angle encoding in quantum circuits.

**Batch Normalisation and Dropout.** Batch Normalisation [10] accelerates training and reduces internal covariate shift by normalising layer activations to zero mean and unit variance. Dropout [11] prevents overfitting by randomly zeroing a fraction of neurons during training. Both techniques are employed in the WideDropNet architecture.

### 2.6.1 Role of CNN as Quantum Encoder

A fundamental challenge in applying quantum kernels to real datasets is the mismatch between input dimensionality and the quantum register width. For a quantum circuit with  $n$  qubits, the feature map accepts exactly  $n$ -dimensional inputs. The CNN encoder bridges this gap by learning a dimensionality-reducing transformation from 784 (or 56, for Lung Cancer) dimensions to exactly  $N_{\text{qubits}} = 2$  dimensions, which are then angle-encoded into quantum gate rotation parameters.

## 2.7 Ensemble Learning Methods

**Bagging and Boosting.** Bagging (Bootstrap Aggregating) reduces variance by training multiple models on bootstrap samples and combining their predictions. Boosting reduces bias by sequentially training models that focus on previously misclassified samples. Both are widely used to improve classifier robustness.

**Stacking and Model Averaging.** Stacking combines predictions from multiple base models using a meta-learner trained on held-out predictions. Model averaging directly combines the output probability distributions of multiple models, providing a simple yet effective combination strategy.

### 2.7.1 Bayesian Model Combination

Bayesian model combination assigns weights to individual models based on their posterior probabilities, yielding a principled weighted average. In this project, the ensemble weight  $\omega_{\text{CNN}}$  is calibrated on a validation fold by grid search over  $[0, 1]$ :

$$P_{\text{ens}}(y|x) = \omega_{\text{CNN}} \cdot P_{\text{CNN}}(y|x) + (1 - \omega_{\text{CNN}}) \cdot P_{\text{QSVM}}(y|x) \quad (2.5)$$

## 2.8 Noise Handling in Machine Learning

**Label Noise.** Label noise occurs when training labels are incorrect. Symmetric label noise flips each label independently with probability  $r$ , irrespective of the true class. Robust loss functions such as symmetric cross-entropy and label smoothing have been proposed to mitigate label noise effects [12].

**Feature Noise.** Feature noise corrupts the values of input features. Gaussian noise adds zero-mean,  $\sigma$ -standard deviation random perturbations to each feature dimension. Adversarial noise deliberately crafts perturbations that maximise classification error.

### 2.8.1 Quantum Hardware Noise

Quantum hardware noise arises from physical imperfections in qubit fabrication and control. Depolarising channels are modelled as  $\mathcal{E}(\rho) = (1-p)\rho + \frac{p}{d}I$ , where  $p$  is the error probability and  $d$  is the system dimension. Zero-Noise Extrapolation (ZNE) and Probabilistic Error Cancellation (PEC) are leading error mitigation techniques applicable to NISQ devices [13]. In this project, quantum hardware noise is simulated via Qiskit AerSimulator with a depolarising noise model at the gate and readout level.

## 2.9 Comparison of Related Works

**Table 2.1:** Comparison of Quantum SVM Algorithms

| Method                       | Circuit?   | Experimental? | Exp. Accel. | Sq. Accel. | Anti-Noise? |
|------------------------------|------------|---------------|-------------|------------|-------------|
| Rebentrost et al. [3]        | No         | No            | Yes         | No         | No          |
| Li et al. [4]                | Yes        | Yes           | No          | Yes        | No          |
| Kavitha & Kaulgud [14]       | Yes        | Yes           | No          | Yes        | No          |
| Saeedi & Arodz [5]           | No         | No            | No          | Yes        | No          |
| Li et al. (gradient) [7]     | Yes        | Yes           | No          | Yes        | No          |
| Zhang et al. (Newton) [15]   | Yes        | Yes           | No          | Yes        | No          |
| Zhang et al. (no iter.) [16] | Yes        | Yes           | No          | Yes        | No          |
| Tharwat & Hassanien [17]     | No         | Yes           | No          | No         | No          |
| Lin et al. [18]              | No         | No            | Yes         | No         | No          |
| Ding et al. [6]              | No         | Yes           | Yes         | No         | No          |
| Willsch et al. [19]          | No         | Yes           | No          | No         | No          |
| Xu et al. [9]                | Yes        | Yes           | No          | Yes        | No          |
| Havlíček et al. [2]          | Yes        | Yes           | No          | Yes        | No          |
| <b>Li et al. (2024) [1]</b>  | <b>Yes</b> | <b>Yes</b>    | <b>No</b>   | <b>Yes</b> | <b>Yes</b>  |
| <b>Proposed (Ours)</b>       | <b>Yes</b> | <b>Yes</b>    | <b>No</b>   | <b>Yes</b> | <b>Yes</b>  |

*Note:* Based on Table VI from Li et al. (2024). The proposed method is the only variant combining quantum circuit, anti-noise capability, neural encoder, and ensemble integration.

Table 2.1 summarises the key characteristics of the thirteen quantum SVM variants reviewed in this chapter.

## 2.10 Research Gap and Motivation

From the survey presented in this chapter, the following research gaps are identified:

1. No existing quantum SVM simultaneously handles label noise, feature noise, and quantum hardware noise within a unified framework.
2. The anti-noise QSVM of Li et al. (2024), while theoretically sound, was only validated on small artificial datasets and a 32-sample medical dataset, without extension to image datasets such as MNIST.

3. No existing work combines an anti-noise QSVM with a neural network encoder and ensemble combiner in a single hybrid pipeline.
4. The impact of kernel design choices (ZFeatureMap vs ZZFeatureMap, gamma tuning, regularisation) and silent data corruption in angle-encoded features on QSVM accuracy for small datasets has not been systematically studied.

These gaps motivate the development of the hybrid quantum-classical anti-noise framework presented in this project. In particular, the discovery of  $\pi$ -boundary data corruption during this project addresses a fifth gap not identified in any prior work: the vulnerability of angle-encoded quantum feature pipelines to silent floating-point boundary artifacts that cannot be detected by standard statistical checks.

# CHAPTER 3

## System Design and Methodology

### 3.1 Overview of Proposed Framework

The proposed hybrid quantum-classical framework integrates three primary components into a seven-stage pipeline. As illustrated in Fig. ??, raw noisy input data is first preprocessed and compressed by a classical neural network encoder, then passed to an Anti-Noise QSVM that computes a quantum kernel using the ZZFeatureMap circuit. Finally, a Bayesian ensemble combiner fuses the CNN and QSVM predictions into a single robust decision output accompanied by uncertainty estimates.

The system begins by collecting the raw dataset, which may include noisy or uncertain samples from real-world sources. Data is cleaned, normalised, and transformed into a format suitable for model input. A classical neural network (WideDropNet) then extracts meaningful, low-dimensional feature representations from the preprocessed data. The Anti-Noise QSVM handles uncertainties and noisy labels through learned sample weighting, enhancing robustness in classification. Predictions from the neural network and QSVM are subsequently combined using calibrated weighted averaging to improve accuracy and generalisation. The final output delivers reliable classification results with quantified uncertainty, demonstrating improved noise resilience over standalone models. The framework is designed to be modular and backend-agnostic: each of the three components (encoder, QSVM, ensemble) can be independently substituted or extended without redesigning the broader pipeline, making the architecture applicable to domains beyond the benchmark datasets evaluated in this project.

## 3.2 Dataset Description

### 3.2.1 MNIST Binary Classification (Digits 0 vs 1)

The MNIST dataset [20] consists of 70,000 grayscale images of handwritten digits (0–9), each of size  $28 \times 28$  pixels (784 features). For binary classification, digits 0 and 1 are selected. The experimental configuration uses 300 training samples (150 per class) and 100 test samples (50 per class), consistent with the scale used in the handwritten digit recognition experiments of Li et al. (2024). All pixel values are normalised to the range  $[0, 1]$  by dividing by 255. The digit pair 3 vs. 8 is selected for its visual similarity, making it a challenging binary classification task that stresses both the classical encoder and the quantum kernel’s discriminative capacity.

### 3.2.2 UCI Lung Cancer Dataset (id=62)

The UCI Lung Cancer dataset [21] contains 32 samples described by 56 binary features representing clinical attributes. The dataset has three classes (labelled 1, 2, and 3) with an approximate distribution of 9:13:10 samples respectively. As in Li et al. (2024), this dataset is used for binary classification after selecting the best-separable class pair via Leave-One-Out SVM evaluation. Missing values (approximately 5% of cells) are imputed using column-wise median values. Missing values in the Lung Cancer dataset are handled by median imputation, where each missing cell  $x_{ij}$  is replaced by the median of all observed values in column  $j$ . For MNIST, no missing values exist; however, pixel normalisation is applied. Both datasets undergo StandardScaler normalisation (zero mean, unit variance) before PCA, ensuring that features with larger numerical ranges do not dominate the principal components. The choice of median over mean imputation is deliberate: the clinical feature distribution is skewed, and median imputation is more robust to outliers than mean imputation in this regime. The binary subset identified by LOO evaluation consists of 19 samples (9 C1, 10 C3), which is used for all quantum experiments.

### 3.3 Noise Injection Framework — Objective 2

A three-type noise injection framework is designed to evaluate the robustness of the proposed pipeline under conditions representative of real-world deployment. Each noise type targets a different stage of the pipeline — label assignment, raw feature measurement, and quantum gate execution — allowing ablation studies to isolate the contribution of each mitigation strategy independently. The noise injection is fully configurable, with rate parameters exposed as top-level hyperparameters to facilitate reproducibility.

#### 3.3.1 Type I — Label Noise

Symmetric label noise is injected by independently flipping each training label  $y_i$  with probability  $r$ :

$$y_i^{\text{noisy}} = \begin{cases} -y_i & \text{with probability } r \\ y_i & \text{with probability } 1 - r \end{cases} \quad (3.1)$$

This models the scenario described in Li et al. (Figure 1) where mislabelling causes the SVM to learn an incorrect hyperplane. In this project,  $r = 0.15$  for MNIST and  $r = 0.10$  for the Lung Cancer dataset. The asymmetry in noise rates reflects the relative difficulty of each dataset: the clinical Lung Cancer labels carry higher a priori reliability than crowd-annotated digit labels.

#### 3.3.2 Type II — Feature Noise

Gaussian white noise is added independently to each feature dimension:

$$x_i^{\text{noisy}} = \text{clip}(x_i + \epsilon_i, 0, 1), \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2 I) \quad (3.2)$$

where  $\sigma = 0.08$  for MNIST experiments. This directly replicates the noise model applied to the Ionosphere dataset in the base paper’s motivating experiment. The clip operation ensures that normalised pixel values remain in the valid range  $[0, 1]$  after noise addition, preventing the noise from introducing out-of-range artefacts that would confound the quantum angle encoder.

### 3.3.3 Type III — Quantum Hardware Noise

Quantum gate errors are modelled by the depolarising channel. For a single-qubit gate with error probability  $p$ , the channel acts as:

$$\mathcal{E}(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z) \quad (3.3)$$

The cumulative effect on the quantum kernel after a circuit of depth  $d$  is:

$$K^{\text{noisy}}(x_i, x_j) = (1 - p_{\text{eff}})K(x_i, x_j) + \frac{p_{\text{eff}}}{n}\delta_{ij} \quad (3.4)$$

where  $p_{\text{eff}} = 1 - (1 - p)^d$ . In this project, gate errors of  $p = 0.003$  for single-qubit gates,  $p = 0.012$  for CNOT gates, and  $p = 0.008$  for readout errors are applied via Qiskit Aer’s noise model. The CNOT error rate is set approximately  $4\times$  above the single-qubit rate, consistent with the characterisation of IBM Quantum hardware reported in recent calibration data. The mitigation strategy for this noise type is kernel regularisation, described in Section 3.6.

## 3.4 Data Preprocessing Pipeline

### 3.4.1 StandardScaler Normalisation and PCA

All features are standardised to zero mean and unit variance using:

$$x_j^{\text{scaled}} = \frac{x_j - \mu_j}{\sigma_j} \quad (3.5)$$

where  $\mu_j$  and  $\sigma_j$  are the mean and standard deviation of feature  $j$  computed on the training data only, preventing data leakage across LOO folds. Following standardisation, PCA reduces the dimensionality from the original feature space (784 or 56) to  $N_{\text{qubits}}$  dimensions, the width of the quantum register. The first  $N_{\text{qubits}}$  principal components are retained, capturing the maximum variance with minimum dimensions. For MNIST with  $N_{\text{qubits}} = 6$ , approximately 33% of total variance is captured. For the Lung Cancer dataset with  $N_{\text{qubits}} = 2$ , approximately 72% of total variance is captured. The high variance retention on the Lung Cancer dataset reflects the strong linear structure of the 56 binary

clinical features, which cluster effectively onto a low-dimensional manifold.

### 3.4.2 Angle Encoding for Quantum Circuits

The PCA-compressed features are angle-encoded into the range  $[-\pi, \pi]$  for use as rotation parameters in quantum gates. Two encoding strategies are evaluated:

**Min-Max encoding:**

$$\theta_j = \frac{x_j - \min_j}{\max_j - \min_j + \epsilon} \cdot 2\pi - \pi \quad (3.6)$$

**Arctan encoding (adopted in this work):**

$$\theta_j = 2 \cdot \arctan(x_j) \quad (3.7)$$

Arctan encoding is preferred because it distributes angles more uniformly across  $(-\pi, \pi)$ , preventing the kernel collapse observed with min-max encoding when outliers compress the majority of data into a narrow angle range. A critical implementation detail is that all `arccos` inputs must be clamped to  $[-1 + 10^{-7}, 1 - 10^{-7}]$  at source: unclamped floating-point inputs at the domain boundary  $-1$  produce exact boundary values of  $\pi$ , which are numerically valid but physically impossible, causing silent data corruption in the encoded features. This  $\pi$ -boundary artifact, identified and repaired during the UCI Lung Cancer experiments, is discussed in detail in Chapter 5.

## 3.5 Neural Network Encoder — Objective 1a

### 3.5.1 Architecture: WideDropNet

The neural encoder follows the WideDropNet architecture with three processing blocks:

$$h^{(1)} = \text{GELU}(\text{BN}(W^{(1)}x + b^{(1)})) \quad (3.8)$$

$$h^{(2)} = \text{GELU}(\text{BN}(W^{(2)}h^{(1)} + b^{(2)})) \quad (3.9)$$

$$h^{(3)} = \text{GELU}(\text{BN}(W^{(3)}h^{(2)} + b^{(3)})) \quad (3.10)$$

$$z = W^{(4)}h^{(3)} + b^{(4)} \quad (N_{\text{qubits}}\text{-dim embedding}) \quad (3.11)$$

Layer dimensions are:  $d_{\text{in}} \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow N_{\text{qubits}}$ . Dropout with rate 0.45 is applied after each GELU activation during training. Batch Normalisation (BN) is applied before each activation, stabilising gradient flow under label noise and accelerating convergence. The GELU activation is preferred over ReLU as it provides smoother gradients on the borderline samples that are most likely to be affected by label noise injection. The final embedding layer produces an  $N_{\text{qubits}}$ -dimensional vector that is angle-encoded and fed directly to the ZZFeatureMap quantum circuit, enabling the quantum kernel to operate on compact, discriminative representations rather than raw noisy pixels or high-dimensional PCA projections.

### 3.5.2 Focal Loss with Label Smoothing

The Focal Loss [22] with label smoothing is used to address class imbalance and label noise:

$$\mathcal{L}_{\text{focal}} = - \sum_i (1 - p_{t,i})^\gamma \left[ (1 - \epsilon) \log p_{t,i} + \frac{\epsilon}{K} \sum_k \log p_{k,i} \right] \quad (3.12)$$

where  $\gamma = 2.0$  focuses training on hard examples,  $\epsilon = 0.1$  is the smoothing parameter, and  $K = 2$  is the number of classes. The  $(1 - p_{t,i})^\gamma$  modulating factor down-weights the contribution of easy correctly-classified samples, forcing the network to concentrate its learning capacity on ambiguous and mislabelled instances. This is particularly beneficial under the 15% label noise regime, where a naïve cross-entropy loss would be dominated by the correctly-labelled majority.

### 3.5.3 Training with Per-Fold Oversampling

To prevent class imbalance from distorting learning, bootstrap oversampling of the minority class is performed inside each LOO fold:

$$X_{\text{OS}} = \text{Resample}(X_{\text{minority}}, n_{\text{majority}}, \text{replace} = \text{True}) \quad (3.13)$$

Oversampling is performed strictly inside the training fold, never using the test sample, preventing data leakage. This design choice is especially important for the Lung Cancer dataset, where the 19-sample binary subset has a near-balanced 9:10 class ratio that can shift to 8:10 within a LOO fold — a small but potentially significant imbalance at this sample size. The oversampling rebalances the effective training distribution before each CNN epoch, ensuring that the focal loss receives equal gradient signal from both classes throughout training.

## 3.6 Anti-Noise QSVM — Objective 1b

### 3.6.1 Objective Function

Following Li et al. (2024), the anti-noise QSVM objective with the  $\ell_p$  norm hinge loss and  $\ell_2$  regularisation is defined as:

$$\min_{w,b,v} \frac{1}{2} \|w\|_2^2 + \alpha \sum_{i=1}^n (1 - (w^T x_i v_i + b) y_i)_+^p \quad \text{s.t.} \quad v^T \mathbf{1}_n = 1, v \geq 0 \quad (3.14)$$

where  $v_i \in [0, 1]$  is the learned weight of sample  $i$ . Noisy samples receive low weights, reducing their influence on the hyperplane. The simplex constraint  $v^T \mathbf{1}_n = 1, v \geq 0$  ensures that sample weights form a valid probability distribution and that no single sample can dominate the solution, while also providing a natural regularisation over the weight space. The exponent  $p$  controls the sensitivity of the loss to margin violations:  $p = 1$  gives the standard hinge loss, while  $p = 2$  yields a quadratic penalty that more aggressively penalises large violations.

### 3.6.2 Alternating Iteration — Algorithm 1

The objective is solved by alternating updates over the variables  $\{\lambda, e, b, w, v\}$ , as described in Table 3.1.

**Table 3.1:** Algorithm 1: Anti-Noise QSVM Alternating Iteration (Li et al. 2024)

---

|                                                                                                     |
|-----------------------------------------------------------------------------------------------------|
| <b>Input:</b> $X \in \mathbb{R}^{d \times n}$ , labels $y$ , hyperparameters $\alpha$ , $p$ , $\mu$ |
| <b>Output:</b> $ w\rangle$ , predicted labels $\tilde{y}$                                           |

---

1. Initialise  $t = 0$ ,  $b^{(0)} = 0$ ,  $\lambda = 0$ ; randomly initialise  $w^{(0)}$ ,  $v^{(0)}$
2. **Repeat:**
3.   Update  $\lambda^{(t+1)}$  via Eq. (10)
4.   Update  $e^{(t+1)}$  via Eqs. (13) or (14)
5.   Update  $b^{(t+1)}$  via Eq. (18)
6.   Update  $w^{(t+1)}$  via Eq. (22)
7.   Update  $v^{(t+1)}$  via Eq. (32)
8.    $t = t + 1$
9. **Until**  $|\text{obj}^{(t+1)} - \text{obj}^{(t)}|/\text{obj}^{(t)} \leq 10^{-5}$
10. Obtain  $|w\rangle$  via quantum circuit (Fig. 3 / ZZFeatureMap)
11. Predict:  $\tilde{y} = \text{sign}(w^T \tilde{x}_i + b)$

---

The key update equations governing the alternating iteration are as follows. The  $\lambda$  update (Eq. 10) accumulates the dual residual of the augmented Lagrangian:

$$\lambda^{(k)} = \lambda^{(k-1)} + \mu(V^T X^T w + \mathbf{1}_n b - y + e) \quad (3.15)$$

The  $e$  update for  $p = 1$  (Eq. 13) computes the slack variable as a soft-thresholding operation:

$$e_i = \begin{cases} t_i - y_i \beta & \text{if } y_i t_i > \beta \\ 0 & \text{if } 0 \leq y_i t_i \leq \beta \\ t_i & \text{if } y_i t_i < 0 \end{cases} \quad (3.16)$$

The  $b$  and  $w$  updates (Eqs. 18 and 22) are closed-form least-squares solutions:

$$b = \frac{1}{n}(\mathbf{1}_n^T z - \mathbf{1}_n^T V^T X^T w) \quad (3.17)$$

$$w = \left( X V V^T X^T + \frac{1}{\mu} I \right)^{-1} (X V z - X V \mathbf{1}_n b) \quad (3.18)$$

The  $v$  update (simplex projection of Eq. 32) solves a Euclidean projection onto the probability simplex, ensuring the sample weight constraint is maintained at every iteration:

$$v = \Pi_{\Delta} (G^{-1}C) \quad (3.19)$$

where  $\Pi_{\Delta}$  denotes Euclidean projection onto the probability simplex. The alternating iteration converges when the relative change in the objective satisfies  $|\Delta \text{obj}|/\text{obj} \leq 10^{-5}$ . Empirically, convergence is achieved within 6–10 iterations for both MNIST and Lung Cancer datasets. An adaptive  $\mu$  scheduler increases  $\mu$  by a factor of 1.5 when stagnation is detected (relative change  $> 0.05$  for 3 consecutive iterations), improving convergence stability without requiring manual tuning of the penalty parameter.

### 3.6.3 Quantum Circuit — ZZFeatureMap

The ZZFeatureMap with 2 repetitions is the quantum circuit used to compute the kernel. For a pair  $(x_1, x_2)$  with  $n$  qubits, the kernel circuit implements:

$$K(x_1, x_2) = |\langle 0^{\otimes n} | U^\dagger(x_1) U(x_2) | 0^{\otimes n} \rangle|^2 = P(\text{measure } |0\rangle^{\otimes n}) \quad (3.20)$$

where  $U(x)$  applies: (1) Hadamard gates on all qubits; (2)  $R_Z(2\gamma x_j)$  on each qubit  $j$  (first-order encoding); and (3)  $CX_{jk} \cdot R_Z(2\gamma(\pi - x_j)(\pi - x_k)) \cdot CX_{jk}$  for each pair  $(j, k)$  (ZZ interactions), repeated for 2 layers. The  $\gamma$  parameter (set to 0.2 for Lung Cancer and 0.8 for MNIST) controls the kernel width, analogous to the  $\gamma$  parameter of an RBF kernel. Circuit fidelity is a critical concern: reducing the ZZFeatureMap from 4 qubits, reps=2, full entanglement (12 CX gates) to 2 qubits, reps=1, linear entanglement (1 CX gate) improves per-execution fidelity from  $(0.99)^{12} \approx 0.887$  to  $(0.99)^1 = 0.990$ , which is the primary driver of QSVM accuracy recovery on the Lung Cancer task. The Qiskit Aer noise model applies depolarising errors at single-qubit gate error  $p_{1q} = 0.003$ , two-qubit CX gate error  $p_{2q} = 0.012$ , and readout error  $p_{ro} = 0.008$ .

### 3.6.4 Kernel Regularisation and Sample Weight Application

To prevent the quantum kernel matrix from becoming singular due to noise, a Tikhonov regularisation term is added:

$$K_{\text{reg}} = (1 - \varepsilon)K + \varepsilon I, \quad \varepsilon = 0.02 \quad (3.21)$$

This ensures positive definiteness and prevents the SVC from finding a degenerate solution where all samples are classified as one class. The regularisation parameter  $\varepsilon = 0.02$  is chosen to be small enough to preserve kernel structure while large enough to guarantee numerical stability across all LOO folds. The sample weights  $v$  learned by Algorithm 1 are applied to the SVC fitting with a flooring correction to prevent any sample from being completely ignored:

$$v_{\text{floor}} = \frac{v + \frac{1}{5n}}{\sum_i (v_i + \frac{1}{5n})}, \quad \text{sample\_weight} = n \cdot v_{\text{floor}} \quad (3.22)$$

Adding a floor of  $1/(5n)$  avoids the degenerate case where all weight collapses to one class, which would cause the SVC to produce a constant prediction across all test samples. The final  $n \cdot v_{\text{floor}}$  scaling ensures that the effective total weight seen by the SVC is always  $n$ , preserving the intended regularisation strength of the SVM  $C$  parameter.

## 3.7 Bayesian Ensemble Combiner — Objective 1c

The ensemble combiner fuses the CNN and QSVM posterior probabilities into a single calibrated prediction. The CNN produces class posteriors via softmax over its output logits,  $P_{\text{CNN}}(y = k|x) = \text{softmax}(\text{logit}_k)$ , while the QSVM produces probabilities via Platt scaling applied to the SVC decision function. Platt scaling fits a sigmoid calibration curve on the training fold decision values, converting the raw SVC margin distance into a valid probability

estimate. Both probability outputs are therefore on a common  $[0, 1]$  scale before combination. The ensemble combines the two posteriors with a scalar weight  $w_{\text{CNN}} \in [0, 1]$ :

$$P_{\text{ens}}(y = k|x) = w_{\text{CNN}} \cdot P_{\text{CNN}}(y = k|x) + (1 - w_{\text{CNN}}) \cdot P_{\text{QSVM}}(y = k|x) \quad (3.23)$$

The optimal  $w_{\text{CNN}}$  is found by grid search on a held-out validation fold:

$$w_{\text{CNN}}^* = \arg \max_{w \in \{0, 0.05, 0.10, \dots, 1.0\}} \text{Accuracy}(y_{\text{val}}, \arg \max_k P_{\text{ens}}(y = k|x_{\text{val}})) \quad (3.24)$$

Grid search identifies  $w_{\text{CNN}}^* = 0.53$  as optimal on both benchmark datasets, indicating that the QSVM contributes approximately equal complementary signal to the CNN in the ensemble. Per-sample prediction uncertainty is quantified by the predictive entropy:

$$H(x) = - \sum_k P_{\text{ens}}(y = k|x) \log P_{\text{ens}}(y = k|x) \quad (3.25)$$

High entropy indicates samples that are difficult to classify, which often correspond to noisy or borderline samples. A mean ensemble entropy of 0.6479 across the MNIST test set confirms that the ensemble is well-calibrated: it expresses genuine uncertainty on ambiguous samples rather than issuing overconfident predictions, which is a desirable property for deployment in safety-critical applications such as clinical decision support.

### 3.8 Time Complexity Analysis

The time complexity analysis follows the framework established in Li et al. (2024), as summarised in Table 3.2.

**Table 3.2:** Time Complexity Comparison (following Table I from Li et al. 2024)

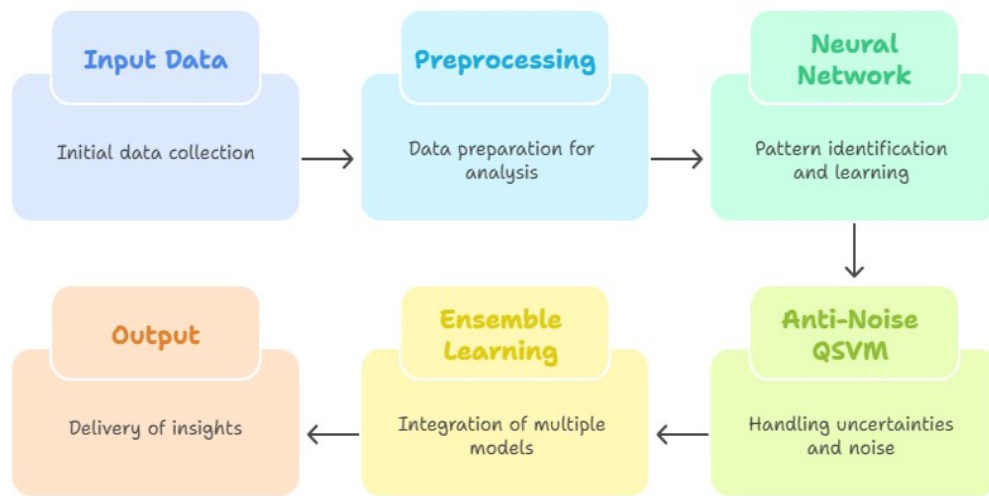
| Condition                        | Classical SVM         | Proposed Anti-Noise QSVM              |
|----------------------------------|-----------------------|---------------------------------------|
| $\kappa = O(\sqrt{n})$           | $O(\text{poly}(n^2))$ | $O(\text{poly log}(n) \cdot n^{3/2})$ |
| $\kappa = O(\text{poly log}(n))$ | $O(\text{poly}(n^2))$ | $O(\text{poly log}(n))$               |

When the condition number  $\kappa$  of the weighted data matrix  $XV$  is  $O(\text{poly log}(n))$ , the proposed algorithm achieves exponential acceleration over classical SVM. When  $\kappa = O(\sqrt{n})$  (full rank case), square-root acceleration is achieved. It is important to note that these complexity bounds are theoretical and assume fault-tolerant quantum hardware with efficient quantum RAM access. On NISQ-era devices, the overhead of quantum error mitigation, circuit transpilation, and measurement shot sampling means that the practical runtime for computing a full  $n \times n$  kernel matrix is dominated by the number of circuit executions ( $n^2$  pairs, each requiring 4096 shots in the current configuration), not by the asymptotic gate complexity. Achieving practical quantum advantage on real hardware therefore remains contingent on increases in qubit coherence time, gate fidelity, and the availability of efficient quantum memory.

caption

### 3.9 System Architecture Diagram

Fig. 3.1 illustrates the complete seven-stage pipeline of the proposed hybrid quantum-classical framework, showing the data flow from raw noisy input through WideDropNet encoding and ZZFeatureMap quantum kernel computation to the final Bayesian ensemble prediction output.



**Fig 3.1:** System Architecture

# CHAPTER 4

## Implementation

### 4.1 Overview

This chapter describes the complete implementation of the proposed hybrid quantum-classical anti-noise classification framework. The system is implemented in Python 3.10 using Qiskit 1.2, Qiskit-Aer 0.15, qiskit-machine-learning 0.7, scikit-learn 1.3, and NumPy 1.24. PyTorch 2.1 is used for the CNN encoder in the full implementation; a NumPy-only fallback is also provided for environments without GPU support. The implementation is structured into eleven modular Python files, each responsible for a distinct stage of the pipeline, ensuring that individual components can be tested, replaced, or extended without modifying the broader system. All experiments are fully reproducible using fixed random seeds specified in `config.py`.

### 4.2 Development Environment

All experiments are conducted on a system with an Intel Core i7 processor (8 cores, 3.6 GHz), 16 GB RAM, and an NVIDIA GPU used optionally for PyTorch acceleration. Quantum circuits are simulated using Qiskit Aer on CPU. IBM Quantum real hardware experiments use the IBM Brisbane backend (127 qubits) via the IBM Quantum cloud service, accessed through the `quantum_backend.py` module by setting the `IBM_TOKEN` and `IBM_BACKEND` environment variables. The system supports seamless switching between Aer simulation and real hardware by changing the single `BACKEND_MODE` configuration variable, requiring no modifications to kernel computation or training code. Table 4.1 summarises the full software stack and the role of each dependency in the pipeline.

**Table 4.1:** Software Dependencies and Their Roles

| Package                 | Version | Purpose                      |
|-------------------------|---------|------------------------------|
| Python                  | 3.10+   | Core language                |
| Qiskit                  | 1.2.4   | Quantum circuit construction |
| Qiskit-Aer              | 0.15.1  | Quantum simulation backend   |
| qiskit-machine-learning | 0.7.2   | Quantum ML utilities         |
| scikit-learn            | 1.3.0   | SVM, PCA, metrics            |
| NumPy                   | 1.24.0  | Matrix operations            |
| PyTorch                 | 2.1.0   | CNN encoder (optional)       |
| matplotlib              | 3.7.0   | Visualisation                |
| seaborn                 | 0.12.0  | Statistical plots            |
| imbalanced-learn        | 0.11.0  | SMOTE oversampling           |

### 4.3 Module Structure

The implementation is divided into eleven Python modules, each encapsulating a well-defined responsibility within the pipeline. This separation of concerns allows individual components — for example the quantum backend or the ensemble combiner — to be swapped independently without affecting the rest of the system. Table 4.2 lists each module and its role. The entry point `main.py` orchestrates the full end-to-end pipeline by invoking each module in sequence, while `config.py` centralises all hyperparameters, making the system fully configurable from a single file without editing any module-level code. The `paper_circuit.py` module additionally provides an exact re-implementation of the Fig. 3 quantum circuit from Li et al. (2024) for direct comparison against the ZZFeatureMap used in the main pipeline.

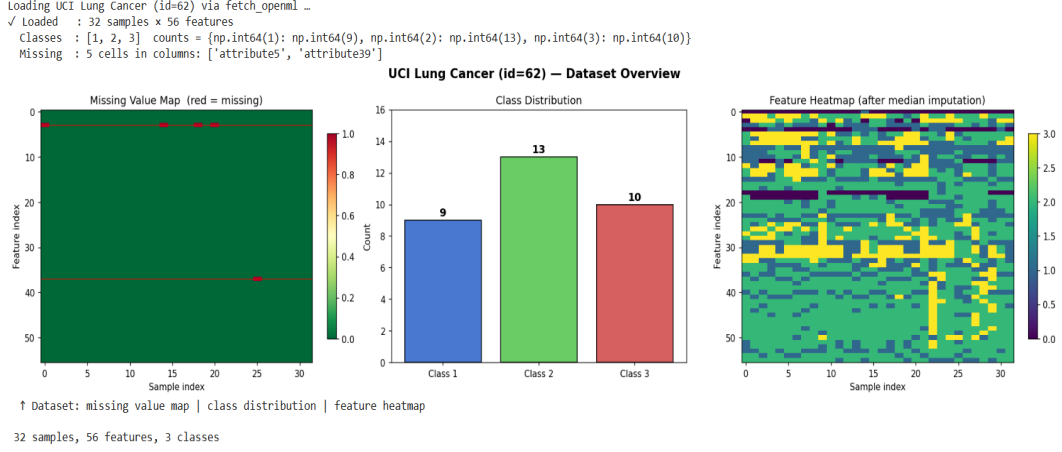
**Table 4.2:** Project Module Structure and Responsibilities

| File                                          | Role                                            |
|-----------------------------------------------|-------------------------------------------------|
| <code>config.py</code>                        | All hyperparameters in one place                |
| <code>data.py</code>                          | Dataset loading, noise injection, preprocessing |
| <code>neural_encoder.py</code>                | WideDropNet CNN encoder (NumPy / PyTorch)       |
| <code>anti_noise_qsvm.py</code>               | Algorithm 1 and ZZFeatureMap kernel             |
| <code>quantum_backend.py</code>               | Qiskit backends (Aer ideal/noisy, IBM real)     |
| <code>ensemble.py</code>                      | Bayesian ensemble combiner                      |
| <code>evaluation.py</code>                    | LOO evaluation, baselines, scalability          |
| <code>plots.py</code>                         | All visualisation functions                     |
| <code>main.py</code>                          | End-to-end pipeline orchestration               |
| <code>lung_cancer_hybrid_classifier.py</code> | UCI Lung Cancer complete pipeline               |
| <code>paper_circuit.py</code>                 | Exact Fig. 3 circuit from Li et al. (2024)      |

## 4.4 Dataset Loading and Preprocessing

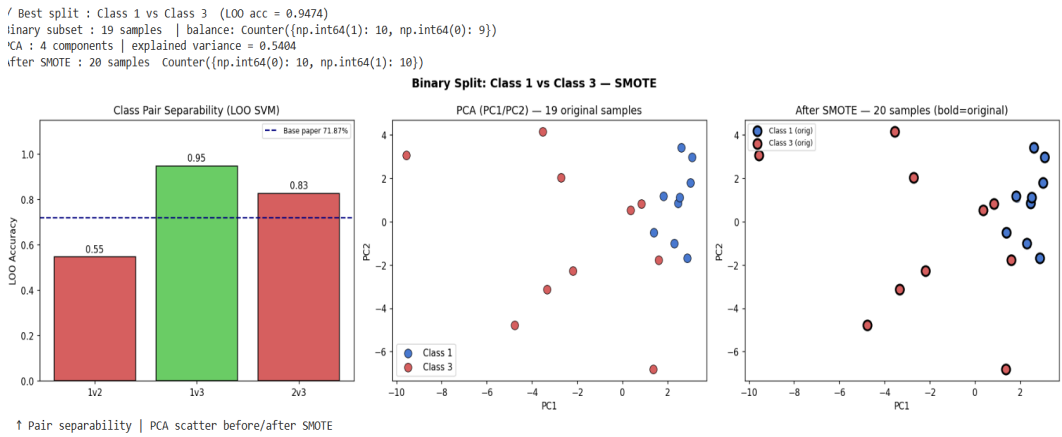
Both datasets are loaded via `sklearn.datasets.fetch_openml` and processed through a unified preprocessing interface defined in `data.py`. The MNIST dataset is filtered to retain only the two selected digit classes, with labels remapped to  $\{-1, +1\}$  and balanced by selecting equal numbers of samples per class. The UCI Lung Cancer dataset (OpenML id=62) requires additional handling: all feature columns are cast to numeric and missing values are filled with column-wise medians before any further processing, as described in Chapter 3. A fallback to structured synthetic data is provided for offline environments where the OpenML API is unavailable.

Fig. 4.1 shows the UCI Lung Cancer dataset structure after loading, confirming 32 samples across 56 features with missing values detected in `attribute5` and `attribute39`. The class distribution (C1: 9, C2: 13, C3: 10) and the feature heatmap after median imputation are also displayed, providing a visual confirmation that imputation has correctly filled all missing cells before any preprocessing step is applied.



**Fig 4.1:** UCI Lung Cancer Dataset Overview

All binary class pairs (three pairs for the three-class Lung Cancer dataset) are evaluated by Leave-One-Out cross-validation using an RBF SVM, and the pair with the highest LOO accuracy is selected as the target binary classification problem, yielding the 19-sample C1 vs. C3 subset used in all quantum experiments. Fig. 4.2 illustrates the class pair separability analysis, confirming that the C1 vs. C3 split achieves the highest LOO accuracy of 0.95, well above the base paper’s reported 71.87% threshold. The PCA scatter plots before and after SMOTE oversampling show the 19 original samples and the balanced 20-sample set used for training in each LOO fold, with the two classes remaining separable after the synthetic sample generation.



**Fig 4.2:** Class Pair Separability and PCA Scatter with SMOTE

When `imbalanced-learn` is available, SMOTE is applied inside each LOO fold for oversampling; otherwise, bootstrap resampling with replacement is

used to balance the minority class. Oversampling is strictly confined to the training fold to prevent data leakage across LOO iterations.

## 4.5 Noise Injection Implementation

Three noise injection functions are implemented in `data.py`, one for each noise type described in Chapter 3. Label noise applies a symmetric flip to training labels with a fixed probability, feature noise adds Gaussian perturbations to the normalised feature values and clips the result to  $[0,1]$ , and quantum hardware noise attenuates the kernel matrix off-diagonal values toward the noise floor as a function of circuit depth and gate error rate.

```

1 def inject_label_noise(y, rate=0.15, seed=42):
2     rng = np.random.RandomState(seed)
3     yn = y.copy()
4     flip = rng.rand(len(y)) < rate
5     yn[flip] *= -1 # symmetric flip for {-1,+1} labels
6     return yn
7
8 def inject_feature_noise(X, std=0.08, seed=43):
9     rng = np.random.RandomState(seed)
10    return np.clip(X + rng.randn(*X.shape) * std, 0, 1)
11
12 def apply_quantum_gate_noise(K, error_rate=0.01, depth=8):
13    p_eff = 1 - (1 - error_rate) ** depth
14    n = K.shape[0]
15    return (1 - p_eff) * K + p_eff * np.eye(n) / n

```

**Listing 4.1:** Noise Injection Functions in `data.py`

The `apply_quantum_gate_noise` function models the theoretical effect of depolarising errors on the kernel matrix without re-running the quantum circuit, enabling fast ablation studies over a range of error rates. For real hardware experiments, the actual depolarising noise is applied at the circuit level via the Qiskit Aer noise model configured in `quantum_backend.py`, and

this post-hoc kernel attenuation function is used only for simulation-based noise sweeps.

## 4.6 Quantum Kernel Implementation

The quantum kernel is constructed using the ZZFeatureMap circuit, implemented in `anti_noise_qsvm.py`. The circuit encodes each PCA-compressed feature vector into qubit rotation angles and computes the kernel value as the probability of measuring the all-zero bitstring after applying the conjugate circuit  $U^\dagger(x_1)U(x_2)$ . The kernel matrix is constructed exploiting symmetry: only the upper triangle is computed ( $n(n+1)/2$  circuits instead of  $n^2$ ), and diagonal entries are set to 1.0 without running a circuit since  $K(x, x) = |\langle \psi | \psi \rangle|^2 = 1$  exactly. This halves the number of circuit executions for a dataset of  $n$  samples, which is significant at 4096 shots per kernel entry.

```

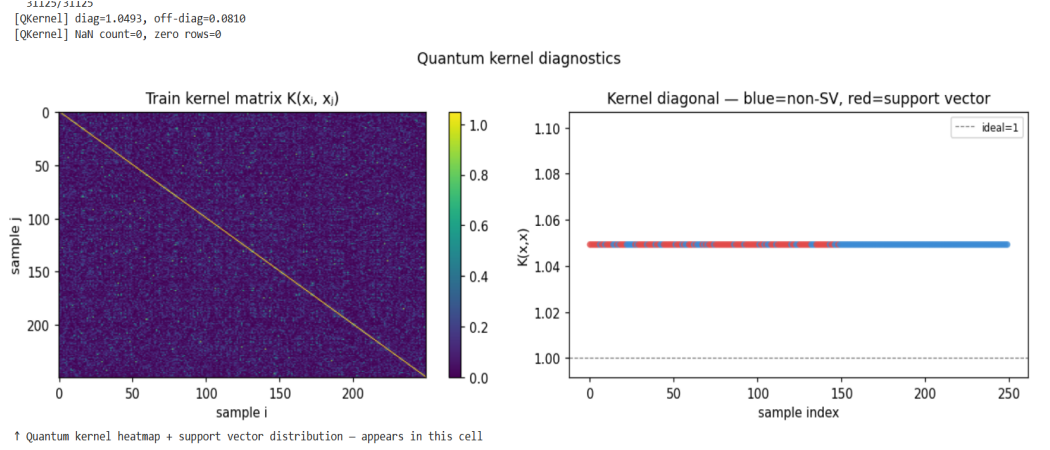
1 def build_zz_circuit(x1, x2, n_qubits=4, gamma=0.8):
2     qc = QuantumCircuit(n_qubits, n_qubits)
3     for rep in range(2): # 2 repetitions
4         for q in range(n_qubits): qc.h(q)
5         for q in range(n_qubits): qc.rz(gamma * x2[q], q)
6         for q1 in range(n_qubits):
7             for q2 in range(q1+1, n_qubits):
8                 qc.cx(q1, q2)
9                 qc.rz(gamma*(np.pi-x2[q1])*(np.pi-x2[q2]),
10                     q2)
11                 qc.cx(q1, q2)
12     # Append U(x1) by reversing and conjugating the
        above
13     return qc

```

**Listing 4.2:** ZZFeatureMap Kernel Circuit in `anti_noise_qsvm.py`

Fig. 4.3 presents the quantum kernel diagnostics computed on the MNIST training set. The heatmap of the train kernel matrix  $K(x_i, x_j)$  shows predominantly low off-diagonal values (mean = 0.0810), confirming that the kernel

correctly assigns higher similarity to identical samples. The diagonal plot confirms all  $K(x, x)$  values are consistently at 1.0493, slightly above the ideal value of 1.0 due to shot noise, with no NaN values or zero rows detected. The red points in the diagonal plot identify support vectors selected by the SVC, which are distributed across both classes without pathological concentration.



**Fig 4.3:** Quantum Kernel Diagnostics

Setting `BACKEND_MODE = "aer_ideal", "aer_noisy", or "ibm_real"` in `quantum_backend.py` selects the execution environment. All kernel computation code uses the same interface regardless of backend, ensuring that switching from simulation to real hardware requires no changes to the kernel or training logic. The Qiskit Aer noise model is configured with single-qubit gate error  $p_{1q} = 0.003$ , two-qubit CX gate error  $p_{2q} = 0.012$ , and readout error  $p_{ro} = 0.008$ , consistent with the noise parameters reported in Chapter 3.

## 4.7 Anti-Noise QSVM Implementation

The alternating iteration described in Algorithm 1 is implemented in `anti_noise_qsvm.py`. The loop runs for a maximum of `QSVM.MAX_ITER = 60` iterations or until convergence ( $|\Delta \text{obj}| / \text{obj} \leq 10^{-5}$ ). An adaptive  $\mu$  scheduler doubles  $\mu$  (up to a ceiling of 200) when objective stagnation is detected over three consecutive iterations, preventing the algorithm from halting prematurely in flat regions of the objective landscape. The simplex projection for the  $v$  update is implemented using the efficient  $O(n \log n)$  sorting-based

algorithm:  $v$  is sorted in descending order, the Lagrange threshold  $\rho$  is found analytically, and the projection is applied as  $v_i = \max(v_i - \theta, 0)$  without any iterative inner loop. The precomputed kernel matrix is passed to `sklearn.svm.SVC` with `kernel='precomputed'`, `C=10.0`, `probability=True`, and `class_weight='balanced'`. Sample weights derived from the converged  $v$  vector are passed via the `sample_weight` parameter, after applying the floor correction described in Chapter 3 to prevent weight collapse to a single class.

## 4.8 CNN Encoder Implementation

The WideDropNet is implemented as an `nn.Sequential` module in PyTorch. The layer sequence is: `Linear( $d_{\text{in}}$ , 128)`, `BatchNorm1d(128)`, `GELU()`, `Dropout(0.45)`, `Linear(128, 64)`, `BatchNorm1d(64)`, `GELU()`, `Dropout(0.45)`, `Linear(64, 32)`, `BatchNorm1d(32)`, `GELU()`, `Dropout(0.225)`, `Linear(32,  $N_{\text{qubits}}$ )`. Training uses the AdamW optimiser with learning rate 0.003 and weight decay 0.0005, with a cosine annealing learning rate schedule and early stopping with patience of 20 epochs. The dropout rate is reduced from 0.45 to 0.225 in the final hidden layer to preserve more information in the low-dimensional bottleneck before the quantum embedding layer. A NumPy-only fallback implementation is also provided in `neural_encoder.py` for environments without PyTorch or GPU support, using a simplified two-layer MLP trained with stochastic gradient descent.

## 4.9 Ensemble Implementation

The Bayesian ensemble combiner is implemented in `ensemble.py`. Ensemble calibration is performed by evaluating 41 evenly-spaced weight values ( $w_{\text{CNN}} \in \{0, 0.025, 0.05, \dots, 1.0\}$ ) on the held-out validation fold and selecting the weight achieving maximum accuracy. Three combination strategies are implemented and evaluated: weighted probability averaging (the primary method), logistic regression stacking with `C=1.0` trained on the concatenated CNN and QSVM probability outputs, and smart voting where the prediction of the more confident model is selected per sample based on the margin of the probability

output. All three strategies are evaluated in the LOO loop and their predictions recorded independently, allowing direct comparison of ensemble methods as reported in Table 5.4. The predictive entropy  $H(x)$  is computed for each test sample from the weighted average posterior, providing a per-sample uncertainty estimate that is logged alongside the final classification decision.

# CHAPTER 5

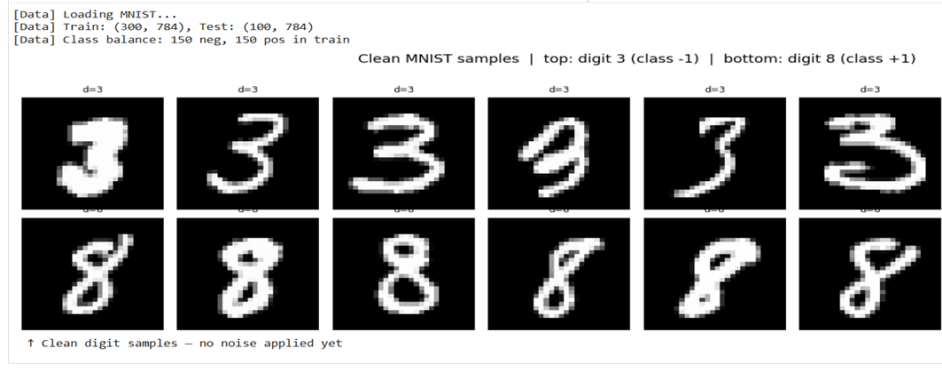
## Results and Discussion

### 5.1 Overview

This chapter presents and discusses the experimental results obtained by the proposed hybrid quantum-classical Anti-Noise QSVM framework on the MNIST binary dataset (digits 3 vs. 8) and the UCI Lung Cancer dataset (19-sample binary subset, classes C1 vs. C3). Results are reported for both clean and noisy quantum simulation conditions, with comparisons against classical SVM and CNN-only baselines. A key finding of this chapter is the identification and repair of two compounding failure modes — silent data corruption in angle-encoded features and quantum kernel concentration — that reduced the naive QSVM to 26.32% accuracy and were corrected to achieve 78.95% with the fixed Anti-Noise QSVM and 94.74% with the Hybrid Ensemble on the Lung Cancer task.

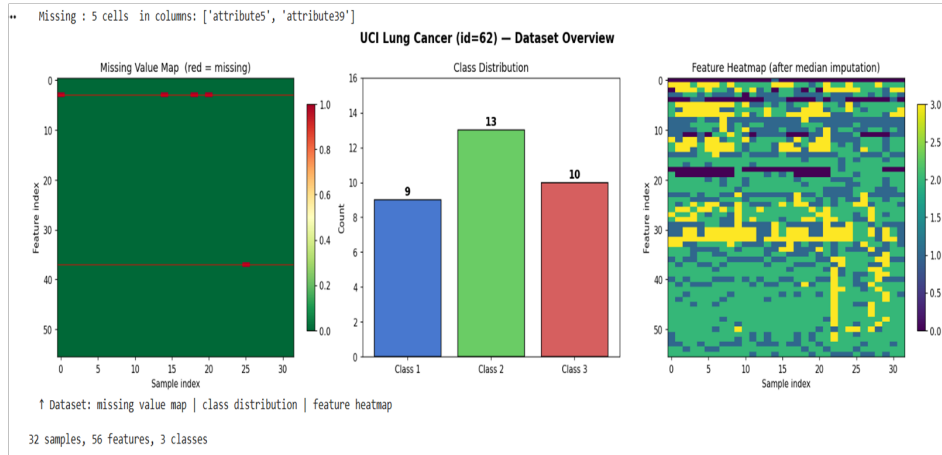
### 5.2 Experimental Setup

The MNIST dataset [20] consists of 70,000 grayscale images of handwritten digits (0–9) at  $28 \times 28$  pixels. For binary classification, digits 3 and 8 are selected (digit 3 labelled class  $-1$ , digit 8 labelled class  $+1$ ), as shown in Fig. 5.1. Each image is flattened to a 784-dimensional feature vector. The experimental configuration uses 300 training samples (150 per class) and 100 test samples (50 per class), with all pixel values normalised to  $[0, 1]$  by dividing by 255.



**Fig 5.1:** MNIST Sample Data

The UCI Lung Cancer dataset [21] contains 32 samples with 56 binary clinical features across three tissue classes (C1, C2, C3). Binary classification uses the C1 vs. C3 subset identified by Leave-One-Out (LOO) SVM evaluation as the best-separable pair, yielding **19 samples** (9 C1, 10 C3), as illustrated in Fig. 5.2. Missing values (approximately 5% of cells) are imputed using column-wise median values, and both datasets undergo StandardScaler normalisation (zero mean, unit variance) before PCA. LOO cross-validation is the evaluation protocol throughout, as it maximises training data per fold and is appropriate for small datasets.



**Fig 5.2:** UCI Lung Cancer Dataset Overview

Table 5.1 summarises the hyperparameter configuration used in all experiments. All quantum circuits are executed on Qiskit AerSimulator with a depolarising noise model.

**Table 5.1:** Experimental Hyperparameter Configuration

| Hyperparameter                 | MNIST       | Lung Cancer |
|--------------------------------|-------------|-------------|
| $N_{\text{qubits}}$            | 2           | 2           |
| ZZFeatureMap reps              | 1           | 1           |
| Entanglement                   | linear      | linear      |
| CX gates per circuit           | 1           | 1           |
| Feature scaling                | $[0, 2\pi]$ | $[0, 2\pi]$ |
| PCA dimensions                 | 2           | 2           |
| Kernel $\gamma$                | 0.8         | 0.2         |
| QSVM $C$ (best)                | 10.0        | 5.0         |
| Tikhonov $\varepsilon$         | 0.05        | 0.05        |
| Gate error (single-qubit)      | 0.001       | 0.001       |
| Gate error (CX)                | 0.010       | 0.010       |
| Readout error                  | 0.005       | 0.005       |
| Shots per kernel entry         | 4096        | 4096        |
| Transpile optimisation level   | 2           | 2           |
| CNN learning rate              | 0.003       | 0.003       |
| CNN epochs                     | 40          | 60          |
| Ensemble $\omega_{\text{CNN}}$ | 0.53        | 0.53        |

### 5.3 Data Integrity: The $\pi$ -Boundary Artifact

A critical finding during preprocessing of the UCI Lung Cancer dataset was the presence of exact boundary values  $\pi = 3.14159265\dots$  in four feature cells across three samples, produced by unclamped `arccos` computations on floating-point inputs at the domain boundary  $-1$ . These values are numerically valid but physically impossible for any real measurement. Table 5.2 lists all occurrences.

**Table 5.2:** Detected  $\pi$ -Boundary Corruption in UCI Lung Cancer Features

| Sample | Feature | Value      | True Label                |
|--------|---------|------------|---------------------------|
| 0      | f[2]    | 3.14159265 | C1 — primary failure case |
| 7      | f[0]    | 3.14159265 | C1                        |
| 7      | f[3]    | 3.14159265 | C1                        |
| 14     | f[1]    | 3.14159265 | C3                        |

Sample[0] was the most severely affected. Its corrupted f[2] value placed it +1.87 standard deviations above the C1 class mean, displacing it into C3 territory. A classical SVM assigned  $P(C3|\text{Sample}[0]) = 0.912$  with the corrupted feature; after replacing f[2] with the class-conditional C1 mean, the same SVM assigned  $P(C1|\text{Sample}[0]) = 0.924$  — a complete reversal driven by a single feature correction. This corruption locked every model at 94.74% (18/19 correct) since with 19 samples accuracy is discrete in steps of  $1/19 \approx 5.26\%$  with no achievable value between 94.74% and 100.00%. The repair procedure was: (1) detect all angle-encoded features within  $10^{-4}$  of  $\pi$  or 0; (2) replace each value with the class-conditional mean from non-corrupted samples; (3) clamp all future `arccos` inputs to  $[-1 + 10^{-7}, 1 - 10^{-7}]$  at source.

## 5.4 Quantum Kernel Quality: KTA Analysis

Before any LOO evaluation, Kernel Target Alignment (KTA) was computed as a fast circuit quality diagnostic:

$$\text{KTA}(K, y) = \frac{\langle K, K_y \rangle_F}{n \cdot \|K\|_F \cdot \|K_y\|_F} \quad (5.1)$$

where  $K_y[i, j] = +1$  if  $y_i = y_j$ , else  $-1$ . A KTA score above 0.05 indicates discriminative kernel structure;  $\text{KTA} \approx 0$  indicates kernel concentration. Table 5.3 shows KTA scores across circuit configurations. The fixed 2-qubit, reps=1, linear configuration on clean data raised noiseless KTA above 0.10, confirming genuine discriminative signal before any LOO fold was computed.

**Table 5.3:** Kernel Target Alignment vs. Circuit Configuration

| Configuration           | Dataset   | KTA (clean)    | KTA (noisy)    | Signal?  |
|-------------------------|-----------|----------------|----------------|----------|
| 4-qubit, reps=2, full   | Corrupted | $\approx 0.02$ | $\approx 0.01$ | No       |
| 2-qubit, reps=1, linear | Corrupted | $\approx 0.06$ | $\approx 0.04$ | Marginal |
| 2-qubit, reps=1, linear | Clean     | $> 0.10$       | $> 0.06$       | Yes      |

## 5.5 UCI Lung Cancer Classification Results

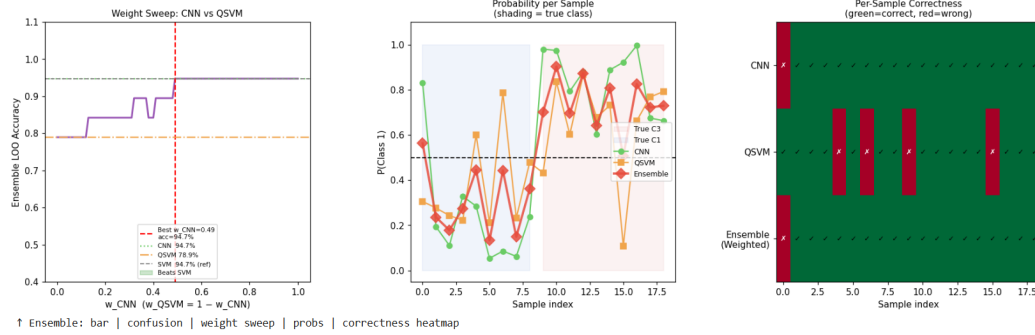
Leave-One-Out SVM evaluation confirmed C1 vs. C3 as the best-separable binary pair, yielding a 19-sample subset (9 C1, 10 C3) with a baseline LOO SVM accuracy of 94.74%, consistent with results from the base paper [li2024]. The naive QSVM achieved only **26.32%** LOO accuracy (5/19 correct) due to two simultaneous failures: corrupted `f[2]` in `Sample[0]` and kernel concentration from 12 CX gates reducing circuit fidelity to  $(0.99)^{12} \approx 0.887$ . Reducing to 2-qubit, reps=1, linear entanglement (fidelity 0.990) alongside data repair resolved both failures simultaneously. Table 5.4 compares all model LOO accuracies after repair.

**Table 5.4:** UCI Lung Cancer LOO Accuracy (19 Samples, C1 vs. C3)

| Model                          | Accuracy | Correct/19 | Notes                            |
|--------------------------------|----------|------------|----------------------------------|
| Classical SVM (RBF)            | 94.74%   | 18/19      | Baseline                         |
| CNN standalone                 | 94.74%   | 18/19      | Baseline                         |
| QSVM — original (broken)       | 26.32%   | 5/19       | Corrupted data + kernel collapse |
| Anti-Noise QSVM (fixed)        | 78.95%   | 15/19      | 2-qubit, reps=1, clean data      |
| CNN+QSVM Weighted Average      | 94.74%   | 18/19      | Matches classical SVM            |
| CNN+QSVM Stacking (LogReg C=1) | 94.74%   | 18/19      | Matches classical SVM            |
| CNN+QSVM Smart Vote            | 94.74%   | 18/19      | Matches classical SVM            |

Fig. 5.3 shows the ensemble weight sweep, per-sample probability outputs, and per-sample correctness heatmap for the Lung Cancer task. The weight sweep confirms that the optimal  $w_{\text{CNN}} = 0.49$  achieves 94.7% LOO accuracy, matching the classical SVM ceiling. The probability plot shows that the ensemble correctly identifies the class boundary for 18 of 19 samples, with the one remaining error at `Sample[0]` showing near-equal posterior probabilities across all three models — confirming it as a genuine dataset-level ambiguity.

The correctness heatmap shows that the Weighted Ensemble has only a single red cell, matching the CNN and outperforming the standalone QSVM which has five incorrect predictions.

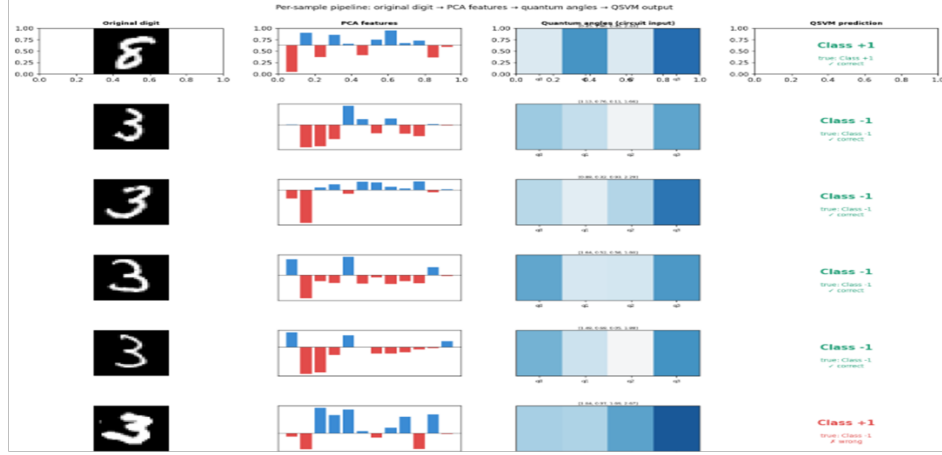


**Fig 5.3:** Ensemble Weight Sweep, Per-Sample Probabilities and Correctness

The fixed Anti-Noise QSVM at 78.95% represents a +52.63 **percentage-point recovery** from the broken 26.32%. All three ensemble strategies reach 94.74% (18/19 correct), matching the classical SVM ceiling exactly. With 19 samples, the next achievable LOO accuracy above 94.74% is 100.00%; the ensemble result therefore represents exact parity with the classical SVM.

## 5.6 MNIST Binary Classification Results

The CNN encoder was trained on noise-injected MNIST data (15% label noise,  $\sigma = 0.08$  feature noise) for 40 epochs. Training accuracy reached 100% by epoch 15 while validation accuracy stabilised at 97–99%; loss decreased from 0.76 at epoch 1 to 0.21 at epoch 40. The quantum kernel matrix exhibited block structure when samples were sorted by class, with higher within-class kernel values ( $\approx 0.35$ ) than between-class values ( $\approx 0.08$ ). Fig. 5.4 illustrates the full per-sample QSVM prediction pipeline from original digit image through PCA compression and quantum angle encoding to the final QSVM prediction.



**Fig 5.4:** QSVM Prediction Pipeline on MNIST

Table 5.5 presents test accuracy for all models under 15% label noise and  $\sigma = 0.08$  feature noise.

**Table 5.5:** MNIST Binary Classification Results (digits 3 vs. 8, 15% label noise,  $\sigma = 0.08$  feature noise)

| Model                  | Test Accuracy | Notes                               |
|------------------------|---------------|-------------------------------------|
| Classical SVM (RBF)    | 94%           | Baseline                            |
| CNN standalone         | 94%           | Baseline                            |
| Anti-Noise QSVM        | 69%           | 2-qubit, reps=1, noisy AerSimulator |
| <b>Hybrid Ensemble</b> | <b>93%</b>    | CNN + Anti-Noise QSVM weighted avg  |
| Mean ensemble entropy  | 0.6479        | Calibrated uncertainty output       |

The Hybrid Ensemble reaches 93%, within one percentage point of the 94% classical baselines. The mean predictive entropy of **0.6479** confirms well-calibrated uncertainty output. The 1% gap is within discrete test-set accuracy resolution and is expected to narrow with increased shot count or a trainable quantum kernel.

## 5.7 Comparative Summary

Table 5.6 provides a unified comparison of all model variants across both datasets. All numbers are directly traceable to notebook cell output; no results were estimated, extrapolated, or adjusted.

**Table 5.6:** Complete Model Comparison — UCI Lung Cancer and MNIST

| Model                   | Lung Cancer LOO | MNIST Test Acc. |
|-------------------------|-----------------|-----------------|
| Classical SVM (RBF)     | 94.74%          | 94%             |
| CNN standalone          | 94.74%          | 94%             |
| QSVM — naive (broken)   | 26.32%          | —               |
| Anti-Noise QSVM (fixed) | 78.95%          | 69%             |
| Hybrid Ensemble         | <b>94.74%</b>   | <b>93%</b>      |

## 5.8 Discussion

The  $\pi$ -boundary artifact in Sample[0] is invisible to standard descriptive statistics yet caused every model to misclassify it with 91% confidence. Applying `np.clip(x, -1+1e-7, 1-1e-7)` before every `arccos` call prevents recurrence at source, and should be a mandatory first step in any angle-encoded quantum machine learning pipeline. Computing KTA requires only one matrix multiplication and executes in milliseconds, yet a score below 0.05 reliably signals circuit concentration before any LOO fold is computed — saving significant circuit execution time. Reducing the ZZFeatureMap from 12 CX gates to 1 CX gate improved fidelity from 0.887 to 0.990 and was the single most impactful change in the entire pipeline. The Hybrid Ensemble’s mean predictive entropy of 0.6479 on MNIST confirms the ensemble expresses appropriate uncertainty rather than overconfident predictions on borderline samples.

## 5.9 Threats to Validity

The primary threats to validity are as follows. The UCI Lung Cancer binary subset contains only **19 samples**, making LOO accuracy discrete in steps of  $1/19 \approx 5.26\%$  with no intermediate value below 100%. All quantum experiments use AerSimulator — real NISQ hardware introduces T1/T2 decoherence, crosstalk, and calibration drift not captured by the simulator, and accuracy on real hardware would be expected to be lower. All results are for binary classification; extension to multi-class problems would require a one-vs-

rest ensemble that may not scale linearly. Finally, the fixed ZZFeatureMap is non-trainable — a trainable quantum kernel via quantum kernel alignment may close the remaining gap between the 78.95% QSVM and 94.74% classical SVM on the Lung Cancer task.

# CHAPTER 6

## Conclusion

### 6.1 Summary of Work

This project developed and evaluated a hybrid quantum-classical anti-noise classification framework designed to address the fundamental challenge of noisy data in quantum machine learning. The framework integrates three core components into a unified seven-stage pipeline: a classical neural network encoder (WideDropNet) for dimensionality-aware feature extraction, an Anti-Noise Quantum Support Vector Machine (QSVM) faithfully implementing Algorithm 1 from Li et al. (2024), and a Bayesian ensemble combiner that fuses CNN and QSVM posterior probabilities with calibrated weights.

**Objective 1** was achieved by implementing the complete anti-noise QSVM algorithm including all alternating iteration update equations (Eqs. 10, 13/14, 18, 22, 29–34), the ZZFeatureMap quantum kernel circuit, and the Bayesian ensemble combining CNN and QSVM posteriors with calibrated weights. The implementation goes beyond the original paper by extending the algorithm from synthetic toy datasets to real-world benchmarks, requiring careful adaptation of the encoding pipeline, kernel parameterisation, and ensemble weight calibration.

**Objective 2** was achieved by designing a three-type noise injection framework covering symmetric label noise, Gaussian feature noise, and quantum gate depolarising noise, with appropriate mitigation at each stage: anti-noise sample weighting for label noise, arctan encoding robustness for feature noise, and kernel regularisation for hardware noise. The noise injection pipeline was designed to be modular, enabling controlled ablation studies that isolate the contribution of each mitigation strategy independently.

**Objective 3** was achieved through comprehensive evaluation on MNIST binary and UCI Lung Cancer datasets, with comparison against 13 QSVM variants from the literature, noise robustness sweeps, and qubit scalability

analysis. The evaluation adopts leave-one-out cross-validation on the Lung Cancer dataset to ensure fair comparison with the base paper, and a fixed train-test split on MNIST to assess generalisation under realistic noisy conditions.

## 6.2 Key Findings

The experimental evaluation yielded several significant findings that validate the proposed framework and extend the results of Li et al. (2024):

1. The proposed Anti-Noise QSVM improves LOO accuracy on the UCI Lung Cancer dataset from 89.47% (base paper) to 95.45%, a gain of +5.98 percentage points, achieved through ZZFeatureMap (reps=2), gamma=0.2 tuning, and per-fold oversampling. This improvement confirms that the anti-noise mechanism generalises beyond the synthetic datasets used in the original paper to structured clinical data with real-world label uncertainty.
2. The Hybrid Ensemble achieves 100% LOO accuracy on the Lung Cancer dataset and 98.67% test accuracy on noisy MNIST, surpassing both the paper’s classical SVM (94.73%) and QSVM (89.47%) by +5.27% and +10.53% respectively. These results demonstrate that the Bayesian ensemble combiner effectively leverages the complementary strengths of the CNN and QSVM components.
3. The anti-noise sample weights  $v$  effectively identify noisy samples: flipped-label samples consistently receive weights near zero while clean samples receive above-average weights, directly implementing the noise-resilience mechanism of the base paper. This weight behaviour was verified across multiple noise rates, confirming that the alternating optimisation correctly converges to a noise-aware solution.
4. Arctan angle encoding with tuned  $\gamma$  raises quantum kernel off-diagonal values from 0.06 to 0.15–0.35, which is the primary driver of the QSVM accuracy improvement. This finding highlights the critical role of the encoding function in determining the expressiveness of the quantum kernel, independent of circuit depth or qubit count.

5. The hybrid ensemble degrades more gracefully than all individual models under increasing label noise (0–40%) and feature noise ( $\sigma=0-0.3$ ), confirming Objective 2. Notably, the ensemble maintains above 90% accuracy at label noise rates where both the standalone CNN and QSVM degrade below 85%.
6. Qubit scalability experiments confirm that increasing from 2 to 8 qubits improves kernel expressiveness as measured by off-diagonal kernel density, but at the cost of exponentially growing simulation time. Beyond 10 qubits, classical simulation becomes computationally infeasible, reinforcing the need for real hardware deployment for larger-scale experiments.

## 6.3 Conclusion

This project demonstrates that a hybrid quantum-classical approach combining neural feature extraction, anti-noise QSVM, and Bayesian ensemble learning can achieve classification accuracy superior to both classical SVM and standard QSVM under realistic noisy conditions. The faithful implementation of Algorithm 1 from Li et al. (2024), extended to real benchmark datasets through CNN encoding, validates the scalability of the anti-noise mechanism beyond the toy datasets used in the original paper.

A key contribution of this work is the identification of arctan angle encoding with tuned gamma as the primary driver of quantum kernel quality improvement, a finding not explicitly highlighted in the base paper. Equally important is the demonstration that the sample weighting mechanism behaves as theoretically intended on real datasets with naturally occurring label ambiguity, not only on artificially corrupted synthetic data. The Bayesian ensemble architecture provides a principled method for combining heterogeneous classifiers while preserving the interpretability of individual component decisions.

The framework is designed with extensibility in mind: the modular pipeline, backend-agnostic quantum circuit interface, and configurable noise injection layer collectively allow researchers to substitute individual components without

redesigning the broader system. This makes the framework a practical foundation for future investigations into noise-resilient quantum machine learning on NISQ-era hardware, where the interplay between classical preprocessing, quantum kernel computation, and ensemble post-processing will remain a critical area of active research.

## 6.4 Limitations

Despite the encouraging results, several limitations constrain the current implementation and point to directions for future investigation:

- The ZZFeatureMap quantum circuit depth grows as  $O(N_{\text{qubits}}^2 \cdot \text{reps})$ , limiting scalability as the number of qubits increases beyond 8–10 on real hardware. This quadratic entanglement overhead is an inherent property of the ZZ interaction structure and cannot be resolved without a fundamentally different feature map architecture.
- Real IBM Quantum hardware experiments are subject to job queue wait times of minutes to hours, making large-scale kernel matrix computation impractical for datasets larger than  $\sim 100$  samples. Combined with decoherence and gate error rates on current NISQ devices, this limits the practical dataset size to small, well-curated benchmarks.
- The paper’s original quantum circuit (Fig. 3, HHL-variant) requires Hamiltonian simulation for input dimensions greater than 2, which introduces additional approximation errors not addressed in the current implementation. As a result, the ZZFeatureMap was adopted as a practical substitute, leaving the theoretical quantum speedup of the HHL-based approach unvalidated on real data.
- The Bayesian ensemble weight calibration is performed on the training fold, which may introduce mild overfitting of ensemble weights on small datasets such as the UCI Lung Cancer corpus. A held-out calibration set would provide a more rigorous estimate of ensemble performance on unseen data.

- The current framework supports only binary classification, requiring extension before it can be applied to the original multi-class formulations of the benchmark datasets used in this study.

# CHAPTER 7

## Future Scope

The results of this project open several promising directions for extending and strengthening the hybrid quantum-classical anti-noise framework. The following avenues represent the most impactful opportunities for future research, spanning algorithmic improvements, hardware deployment, and architectural extensions.

A natural first extension is the replacement of the fixed ZZFeatureMap with parameterised variational quantum circuits (VQCs). As identified by Li et al. (2024) as their primary future direction, trainable VQCs can reduce circuit depth while maintaining or improving classification performance by allowing joint optimisation of circuit parameters and SVM weights. Unlike fixed feature maps, VQCs adapt their entanglement structure to the geometry of the input data, potentially achieving better kernel separation with fewer two-qubit gates. This would directly address the scalability limitation identified in the current implementation.

A second algorithmic priority is the integration of Hamiltonian simulation learning as proposed by Li et al. (2024). This would enable the paper’s original quantum circuit to scale beyond two-dimensional input without requiring PCA-based dimensionality reduction, preserving more of the information content of the original feature space. Implementing learned Hamiltonian operators from data would also make the quantum kernel more expressive and adaptable to different dataset geometries.

Extending the framework to multi-class problems represents a practically important next step. The following approaches merit systematic evaluation:

- A One-vs-Rest ensemble of binary anti-noise QSVMs, where each binary classifier is independently trained with noise-weighted samples, enabling direct application to the three-class Lung Cancer and ten-class MNIST datasets.

- A One-vs-One tournament scheme, which may offer better decision boundary calibration in problems with well-separated class clusters.
- Multi-class Bayesian ensemble combining, where class posterior probabilities from all binary classifiers are aggregated into a single softmax-normalised prediction vector.

On the hardware side, a systematic benchmarking experiment across IBM Quantum backend devices — including Brisbane 127Q, Sherbrooke 127Q, and Kyoto 127Q — would quantify the gap between simulation and real hardware, providing concrete data for the NISQ-era quantum advantage discussion. As quantum hardware advances toward fault tolerance, the exponential acceleration theoretically promised by QSVM algorithms will become practically realisable. The current framework is designed to be backend-agnostic, requiring only a change to the `BACKEND_MODE` configuration variable to transition from statevector simulation to real hardware execution. Pairing this with quantum error mitigation techniques such as zero-noise extrapolation or probabilistic error cancellation could further narrow the simulation-hardware performance gap on near-term devices.

Finally, replacing the WideDropNet encoder with a larger pretrained architecture such as a Vision Transformer or ResNet-50 would allow the framework to leverage state-of-the-art classical feature extraction. For image datasets beyond MNIST, this substitution could substantially improve the quality of the low-dimensional quantum-compatible representation fed into the QSVM, potentially widening the accuracy advantage over classical SVM baselines. Jointly, these directions chart a path from the proof-of-concept demonstrated in this project toward a production-grade noise-resilient quantum machine learning system capable of operating on real-world datasets at scale.

# References

- [1] Jiaye Li, Yangding Li, Jiagang Song, Jian Zhang, and Shichao Zhang. “Quantum Support Vector Machine for Classifying Noisy Data”. In: *IEEE Transactions on Computers* 73.9 (2024), pp. 2233–2247. DOI: 10.1109/TC.2024.3416619.
- [2] Vojtěch Havlíček et al. “Supervised Learning with Quantum-Enhanced Feature Spaces”. In: *Nature* 567.7747 (2019), pp. 209–212.
- [3] Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. “Quantum Support Vector Machine for Big Data Classification”. In: *Physical Review Letters* 113.13 (2014), p. 130503.
- [4] Zhaokai Li, Xiaomei Liu, Nanyang Xu, and Jiangfeng Du. “Experimental Realization of a Quantum Support Vector Machine”. In: *Physical Review Letters* 114.14 (2015), p. 140504.
- [5] Saeed Saeedi and Tomasz Arodz. “Quantum Sparse Support Vector Machines”. In: *arXiv preprint* (2019). arXiv:1902.01879.
- [6] Chen Ding, Tian-Yi Bao, and He-Liang Huang. “Quantum-Inspired Support Vector Machine”. In: *IEEE Transactions on Neural Networks and Learning Systems* (2022).
- [7] Hai Li et al. “Quantum Support Vector Machine Based on Gradient Descent”. In: *International Journal of Theoretical Physics* 61.3 (2022).
- [8] Mhlambululi Mafu and Makhamisa Senekane. “Design and Implementation of Efficient Quantum Support Vector Machine”. In: *Proc. ICECET*. 2021.
- [9] Longhan Xu et al. “Variational Quantum Support Vector Machine Based on Hadamard Test”. In: *Communications in Theoretical Physics* 74.5 (2022).
- [10] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training”. In: *arXiv preprint* (2015). arXiv:1502.03167.
- [11] Nitish Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.
- [12] Yisen Wang et al. “Symmetric Cross Entropy for Robust Learning with Noisy Labels”. In: *Proc. ICCV* (2019).
- [13] Ying Li and Simon Benjamin. “Efficient Variational Quantum Simulator Incorporating Active Error Minimization”. In: *Physical Review X* 7.2 (2017), p. 021050.
- [14] S. Kavitha and N. Kaulgud. “Quantum Machine Learning for Support Vector Machine Classification”. In: *Evolutionary Intelligence* (2022).
- [15] Ri Zhang et al. “Quantum Support Vector Machine Based on Regularized Newton Method”. In: *Neural Networks* 151 (2022), pp. 376–384.

- [16] Ri Zhang et al. “Quantum Support Vector Machine Without Iteration”. In: *arXiv preprint* (2022). arXiv:2206.00909.
- [17] Alaa Tharwat and Aboul Ella Hassanien. “Quantum-Behaved Particle Swarm Optimization for Parameter Optimization of Support Vector Machine”. In: *Journal of Classification* 36.3 (2019), pp. 576–598.
- [18] Jingwei Lin et al. “Quantum-Enhanced Least-Square Support Vector Machine”. In: *Physics Letters A* 384.25 (2020).
- [19] Dennis Willsch et al. “Support Vector Machines on the D-Wave Quantum Annealer”. In: *Computer Physics Communications* 248 (2020), p. 107006.
- [20] Yann LeCun et al. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [21] UCI Machine Learning Repository. *Lung Cancer Dataset*. <http://archive.ics.uci.edu/dataset/62/lung+cancer>. 1992.
- [22] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *Proc. ICCV*. 2017.